Show-Harness

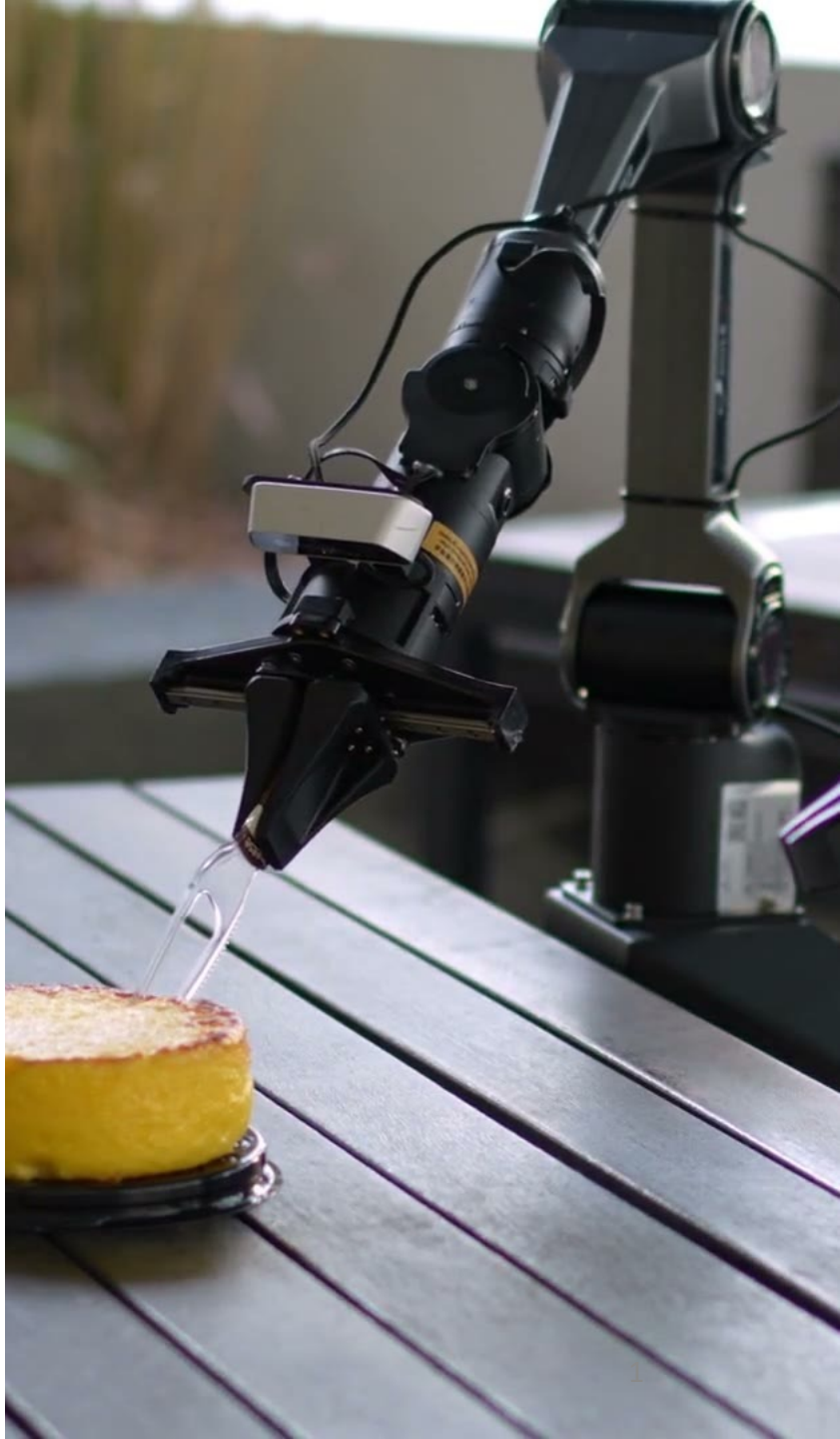


Just a VLM Agent Can Play Robots

Yanzhe Chen
Advisor: Prof. Mike Zheng Shou
SEAIC @ Lorong AI, 30 Oct 2026

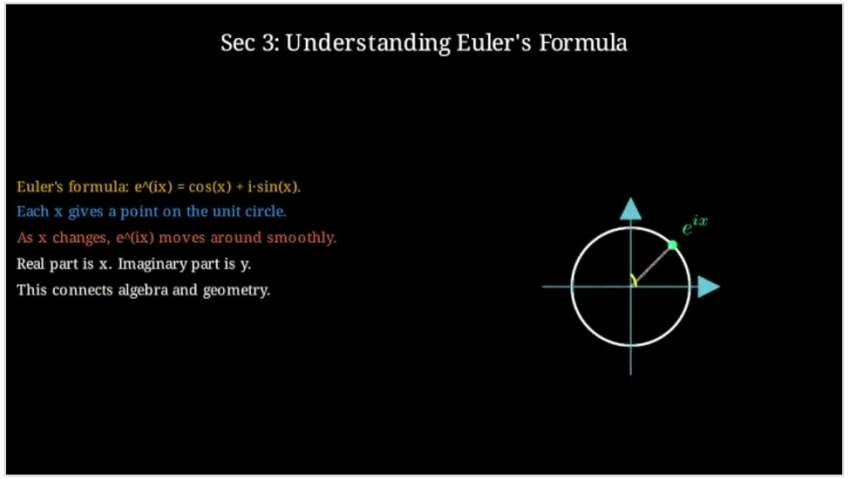
Show

NUS National University of Singapore



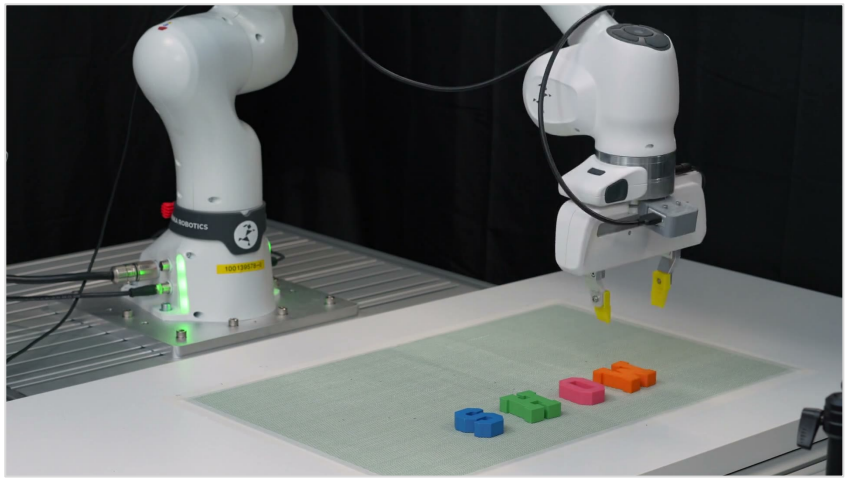
About Me

- I am a PhD student at **Show Lab, NUS**
- Today I am giving this talk on behalf of my advisor, **Professor Mike Shou**
- I work on **AI agents**: AI models that finish tasks step by step, using tools
- For example, to book a flight, an agent uses the web browser as a tool: it searches, compares prices, and fills in the form



Code2Video (ICML 2026)

an agent that makes teaching videos

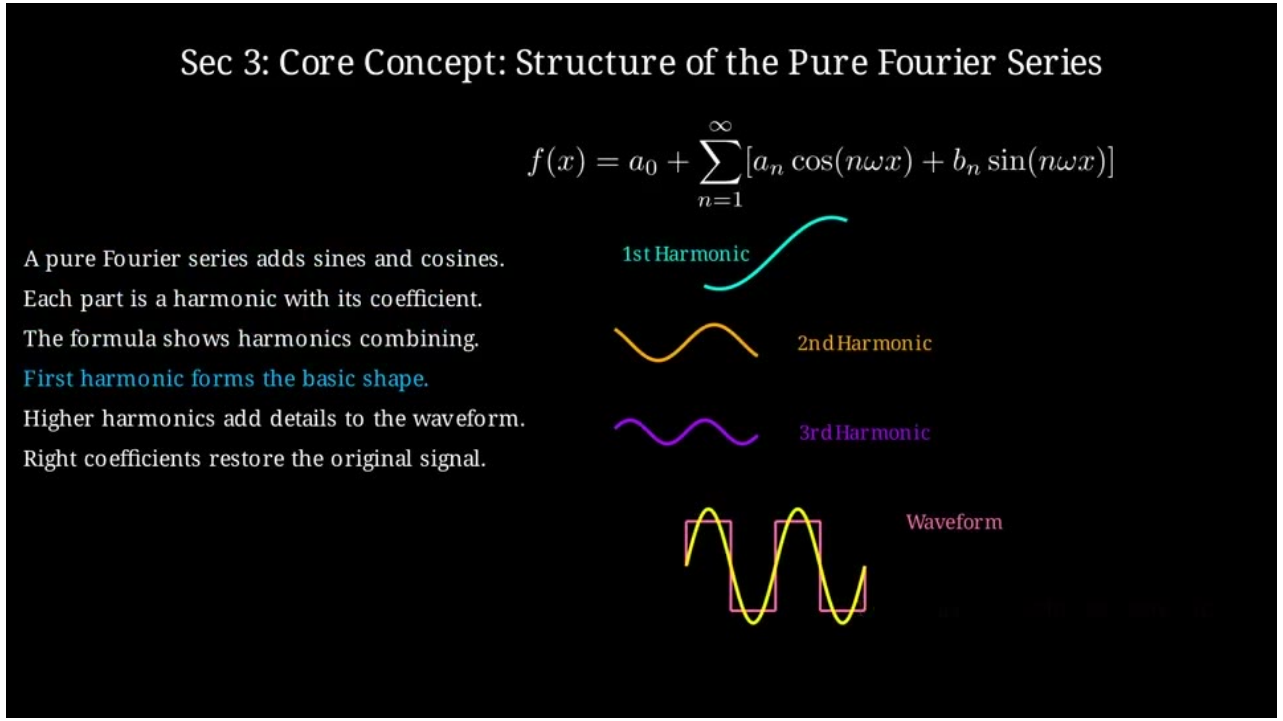


Show-Harness (2026)

an agent that plays robots

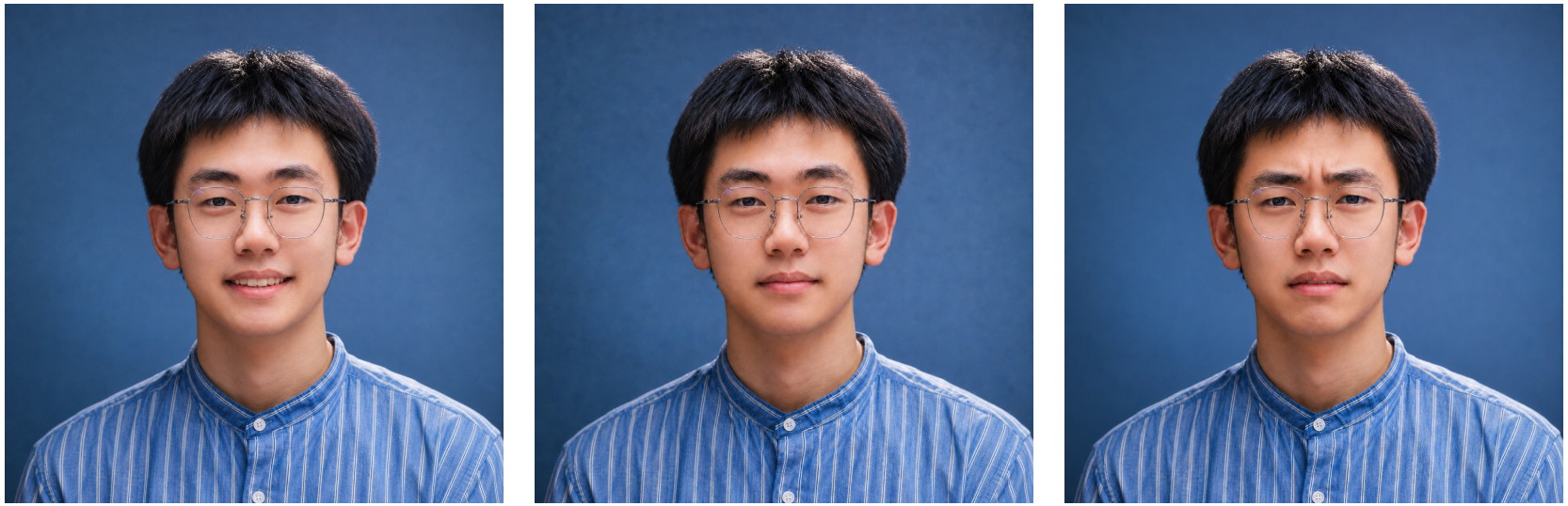
A Simpler Agent: Code2Video

- Today's main topic is **Show-Harness**, our agent for robots
- Firstly, I would like to show you **Code2Video**. It is a simpler agent, showing **what an agent is**
- Code2Video turns a learning topic, like the *Fourier series*, into a full teaching video by itself
- It started with a funny story I had in my first year here ☺



Code2Video: Why Teaching Videos?

- When I came to Singapore for my PhD, I found it a wonderful place for research
- But classes were hard for me. Reading English is easy, and speaking is OK
- **Listening** is the hardest, especially Singlish. It sounds like singing, but I miss the words!
- So in the Q&A, please speak a little slowly for me ☺

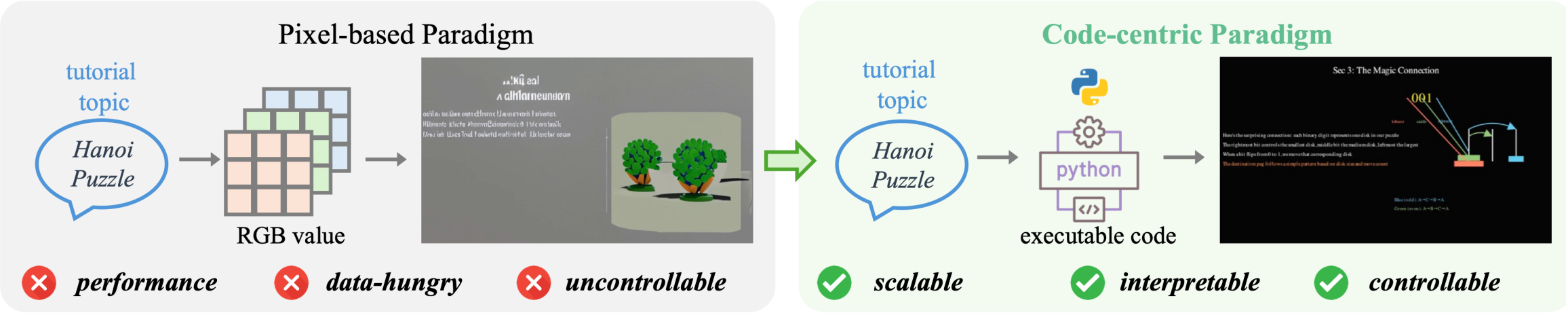


Read: Easy → Speak: OK → Listen: Hard

Code2Video: Let the Model Write Code

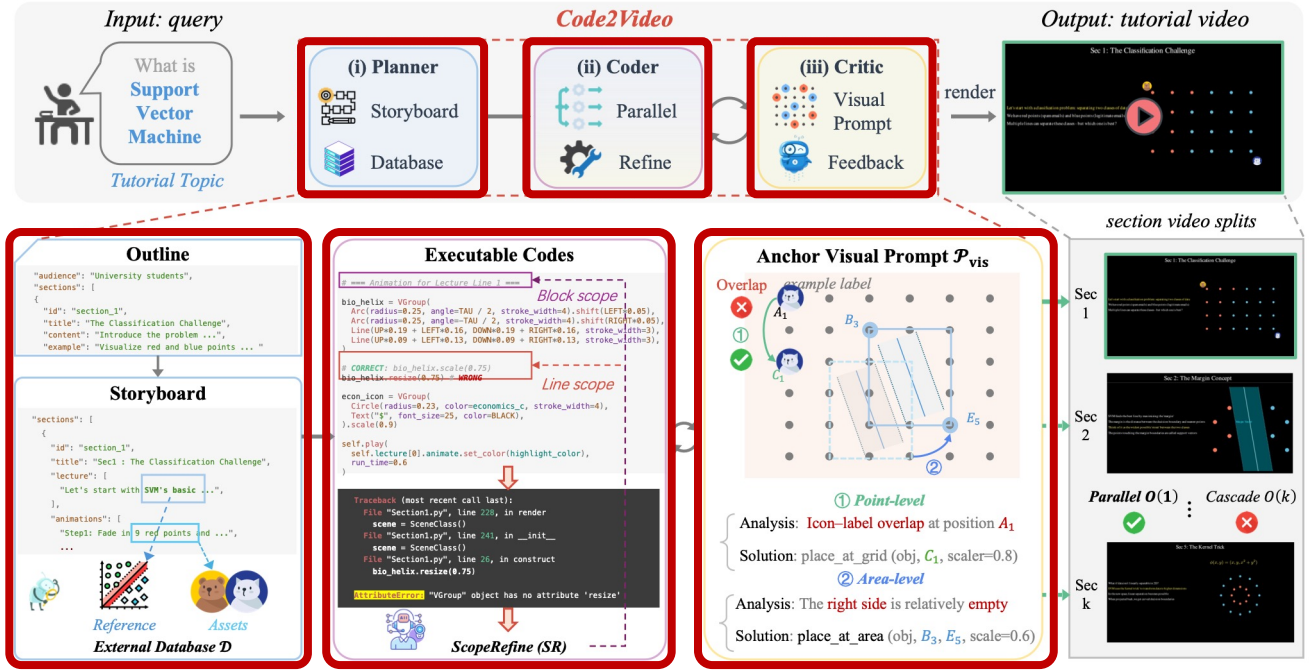
- For me, seeing helps more than listening
- So I wondered: **can AI models directly make teaching videos?**
- Simple Way: Diffusion Models draw pixels, but they often get **formulas wrong** and **cannot generate long videos well**
- Code2Video **writes Python code**, and the code draws the video exactly

Code2Video website



Code2Video: An Agent with a Harness

- But one prompt to the model is not enough: the code can crash, and the layout can be messy
- So a **planner** makes a storyboard, a **coder** writes the code, and a **critic** checks the video
- Together, they make an **agent**. Everything around the model that helps it is the **Harness**
- Please remember this word. Our robot work is called **Show-Harness**



Sec 1: Introduction - What Are Large Language Models?

LLMs are AI that process human language.
They work in chatbots, translation, and content tools.
LLMs learn, not just follow fixed rules.

Sec 1: Introduction: Decomposition of Periodic Signals

Periodic signals repeat patterns.
Complex waves use simpler functions.
Can complex sound form from simple parts?

Sec 1: Introduction to Pi (π): The Circle Connection

A circle has diameter and circumference.
Pi equals circumference divided by diameter.
This ratio is constant for any circle.
Let's measure a pizza as a circle.
Pi connects all circles.
It applies to every circle size.

Sec 1: Introduction & Prerequisite Concepts

A curve is usually a one-dimensional object.
A line extends in one dimension.
A square covers two dimensions.
Can a line fill a square?
Let's explore this question.

Code2Video

Video Generation by Codes

https://github.com/showlab/Code2Video

All these videos were made by our agent

Sec 1: The Towers of Hanoi Challenge

Towers of Hanoi puzzle - a classic problem with a mathematical solution
Pegs labeled A, B, and C, with disks of different sizes
Goal: Move all disks from peg A to peg C
Rule1: Move only one disk at a time
Rule2: Never place a larger disk on a smaller one

Sec 1: Introduction: Linking Algebra to Geometry

Can numbers describe circles?
Math links exponentials with circular shapes.
Imagine a point moving around a circle.

Sec 1: Neural Networks – The Foundation

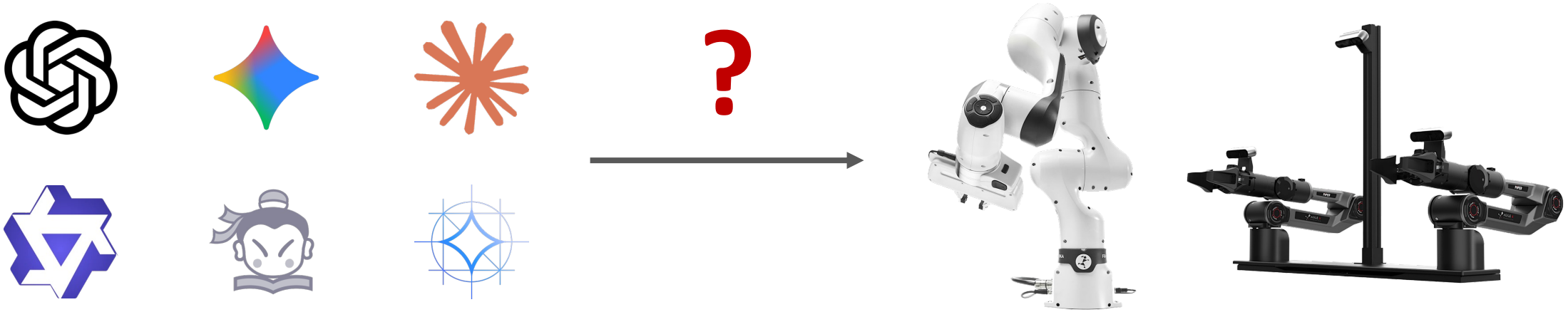
Neural networks mimic the human brain.
They use simple units called neurons.
Neural networks have input, hidden, output layers.
Neurons connect and share information.
Connections allow data flow and processing.

Sec 1: Introduction: Growth Patterns in Nature

Growth patterns are common in nature and life.
Rumors spread fast.
Populations quickly grow.
Resources can run out.

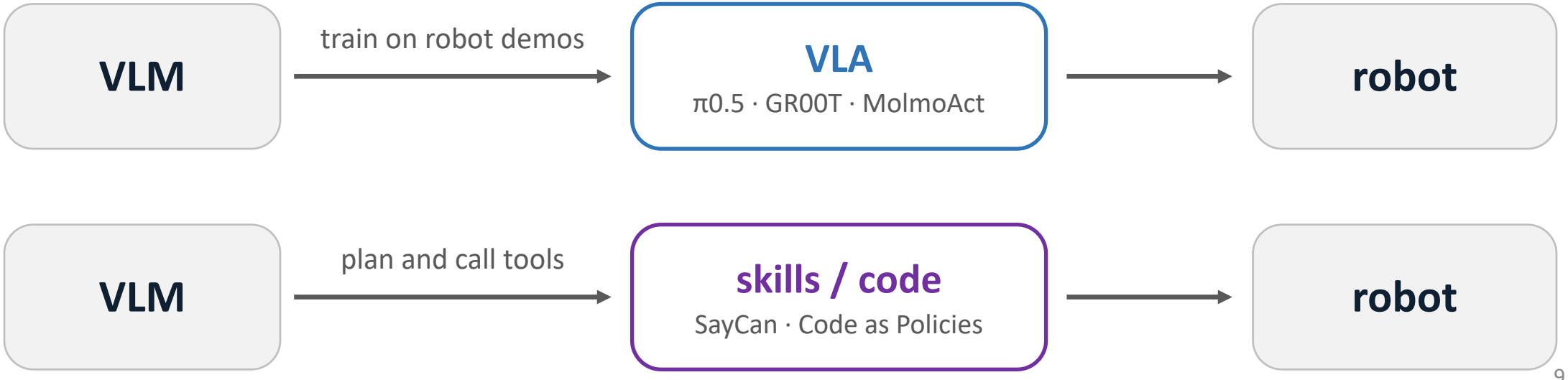
From Videos to Robots

- Now, let's come back to today's topic: **robots**
- Code2Video works on a computer. Can an agent also control a robot in the real world?
- For robots, we use a **Vision-Language Model (VLM)**, like GPT or Gemini
- It looks at images (Input) and answers in words (Output)
- A VLM knows a lot about objects and tasks, but it does not know **the body it is using**



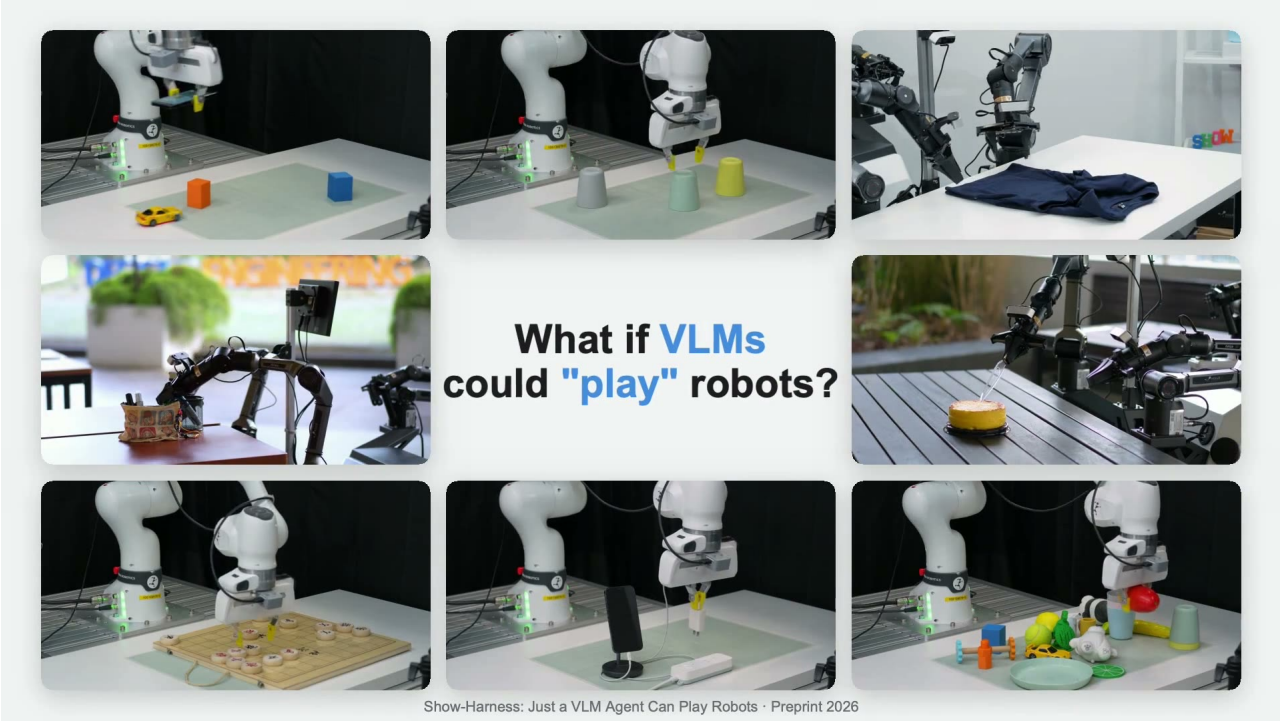
How Robots Use VLMs Today

- Today, robots use VLMs in two common ways
- Way one is a **Vision-Language-Action (VLA)**: we train a VLM on many robot demos, so it outputs the exact numbers that move the arm, many times each second
- Way two is an **agent with tools**: the VLM plans, then calls ready-made skills, like “pick up the cup”, or writes short code that uses them
- Both work well, but each has a cost. VLAs need **new demos**: data hungry. Agents need **ready-made skills**, built by engineers



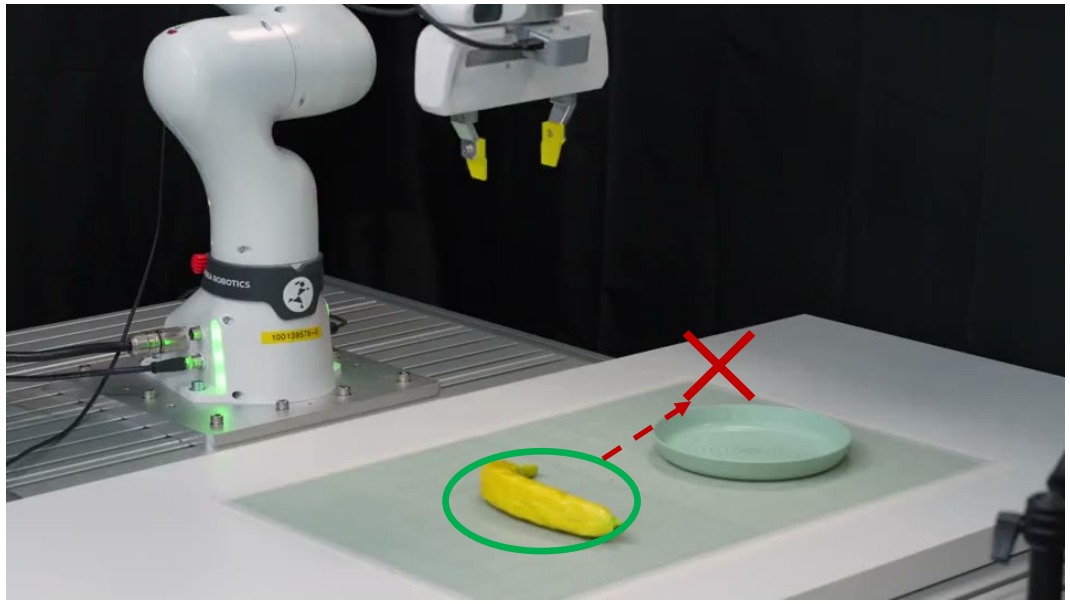
Our Question

- So in April this year, we wanted to answer a simple question
- Can a VLM **play a robot directly**, like a gamer with a gamepad? 🎮
- We wanted another way, a more **easy pipeline to adapt a VLM** 🖼️
- The VLM decides **each move**, and the robot's own control software drives the motors



First Try: Ask for Numbers

- The simplest way is to ask the VLM for numbers
- In April, we asked **Gemini 3.1 Pro** for the gripper's target position. The plan was good, but **the numbers were not accurate enough**
- In this illustration, the predicted point is a few centimeters away from the banana
- Anthropic saw the same in July: Claude's direct control worked only **0 to 5.5%** of the time



```
VLM output:
move_to(x=0.43,
        y=-0.12, z=0.08)
```

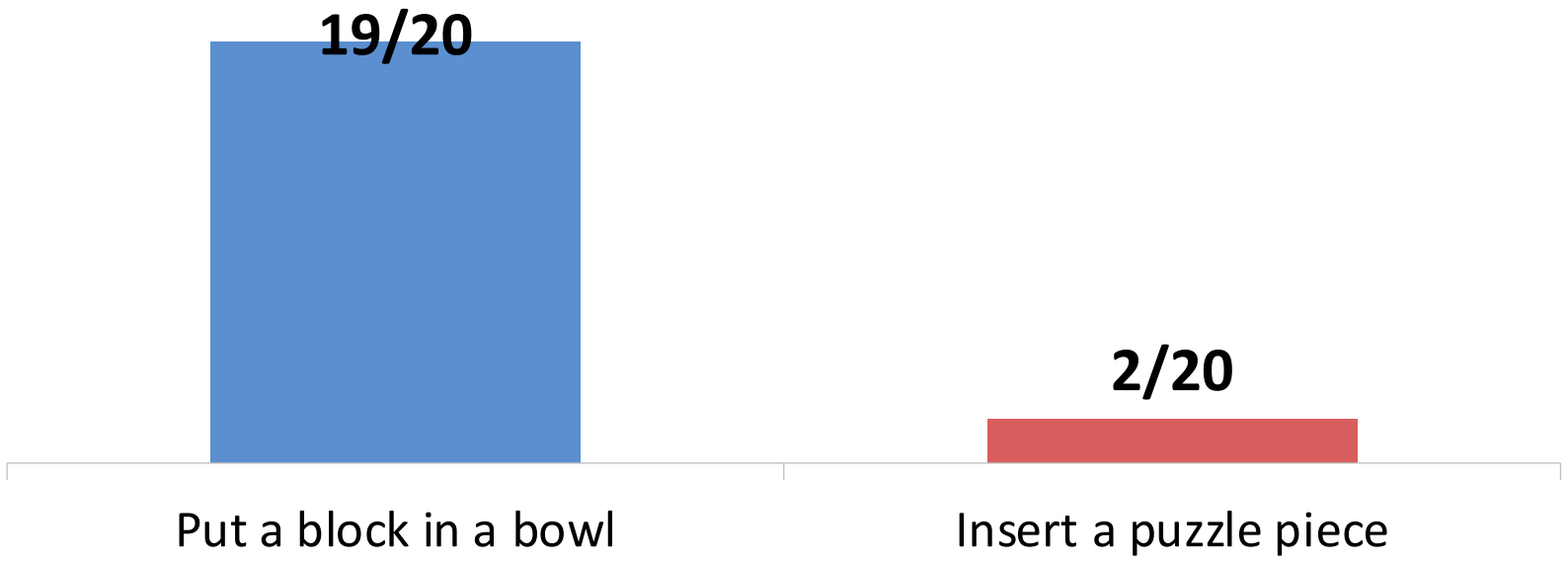
○ banana ✕ predicted target

(illustration)

Anthropic, "Claude plays robotics", Jul 9, 2026 (direct control on LIBERO-style tasks)

September 2026: GPT-6 Astra

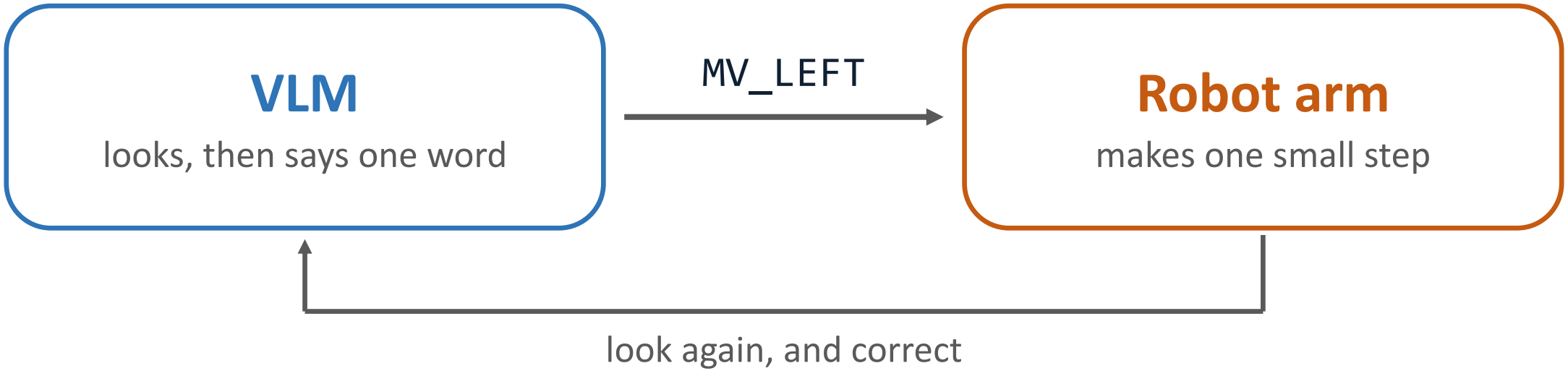
- Then in September, OpenAI released **GPT-6 Astra**. It can output gripper positions directly
- RoboCurve tested it on real robot arms: 19 out of 20 for a block in a bowl, but only 2 out of 20 for a puzzle piece
- So easy moves work now, but **precise moves** are still hard
- Why are exact numbers so hard, even for a strong model? Let us try it ourselves



openai.robocurve.org/gpt-6-astra (Sep 4, 2026; bimanual YAM arms, 20 trials per task)

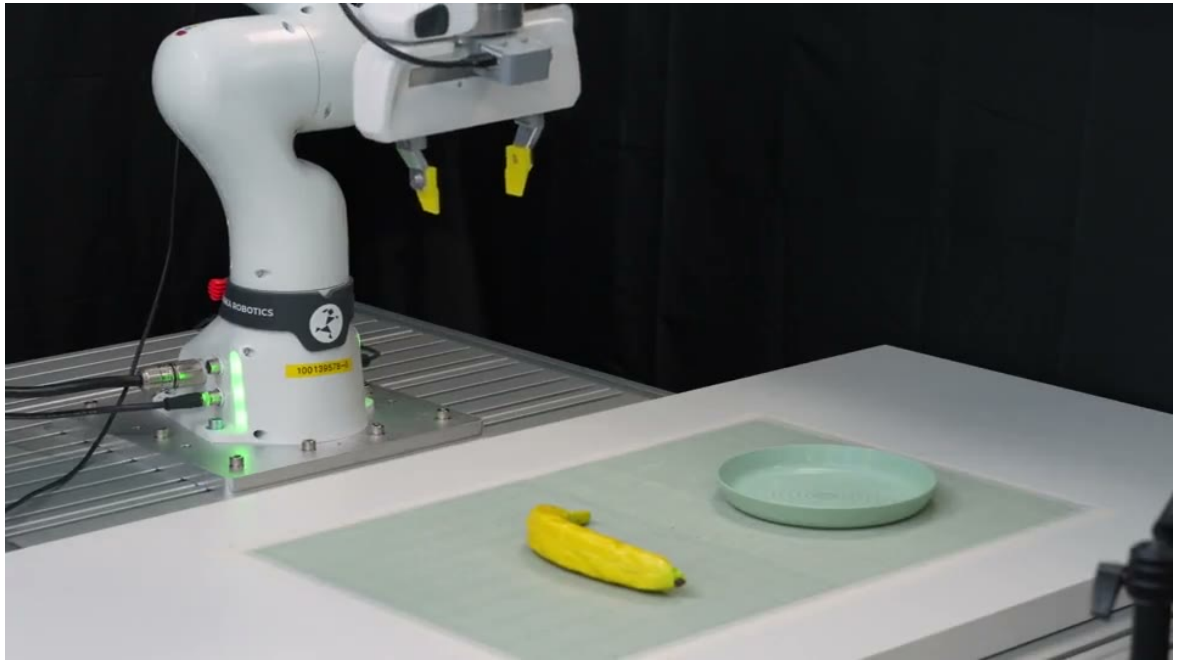
A Game: Why Are Numbers Hard?

- Please close your eyes and put your finger **12 centimeters** left of your phone. Hard, right?
- Now try: “a bit left, a bit more, stop.” **Much easier!** We move a little, look, and correct
- Let us play a small game ☺



A Game: The Rules

- May I have **one volunteer**, please ☺
- You will be the **robot arm**, and I will be the **VLM**
- During the game, please keep your eyes closed. **Only the VLM can see**
- Your task is to put the **Mouse** on the **Mac**, the similar task as our robot here



Our robot, before the task starts

A Game: Robot Words

- I will only say these robot words. Each word means **one small move**, about one hand wide
- **One word, one step.** After each step, please stop and wait for my next word

Word	What you do	On a real robot
MV_FWD	move forward, away from you	move forward 2 centimeters
MV_BACK	move back, toward you	move back 2 centimeters
MV_LEFT	move to your left	move left 2 centimeters
MV_RIGHT	move to your right	move right 2 centimeters
GRASP	close your hand and hold	close the gripper
RELEASE	open your hand	open the gripper
DONE	stop, the task is finished	the task is finished

Real robots also move up and down, and rotate

A Game: Let Us Play

- Now, I will say **one robot word at a time**, and I look at your hand after every step

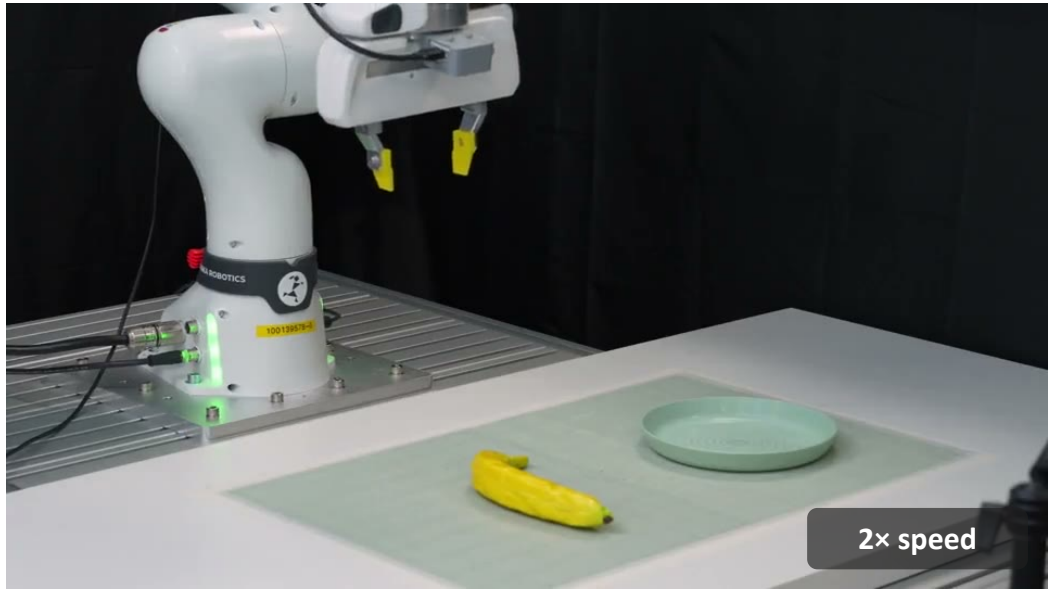
One word, then look

```
MV_FWD … MV_FWD … MV_LEFT …
GRASP … MV_RIGHT … MV_RIGHT …
RELEASE … DONE
```

for example: I choose each word after I look

A Game: What Did We Learn?

- We never planned the whole path. Small steps, with a look after each one, **got us there**
- The “robot” never opened its eyes, and the “VLM” never said a number
- Like code in Code2Video, the above action words are easy for the VLM model to use
- **This is the core motivation of Show-Harness**



Our robot does a similar task, one word at a time

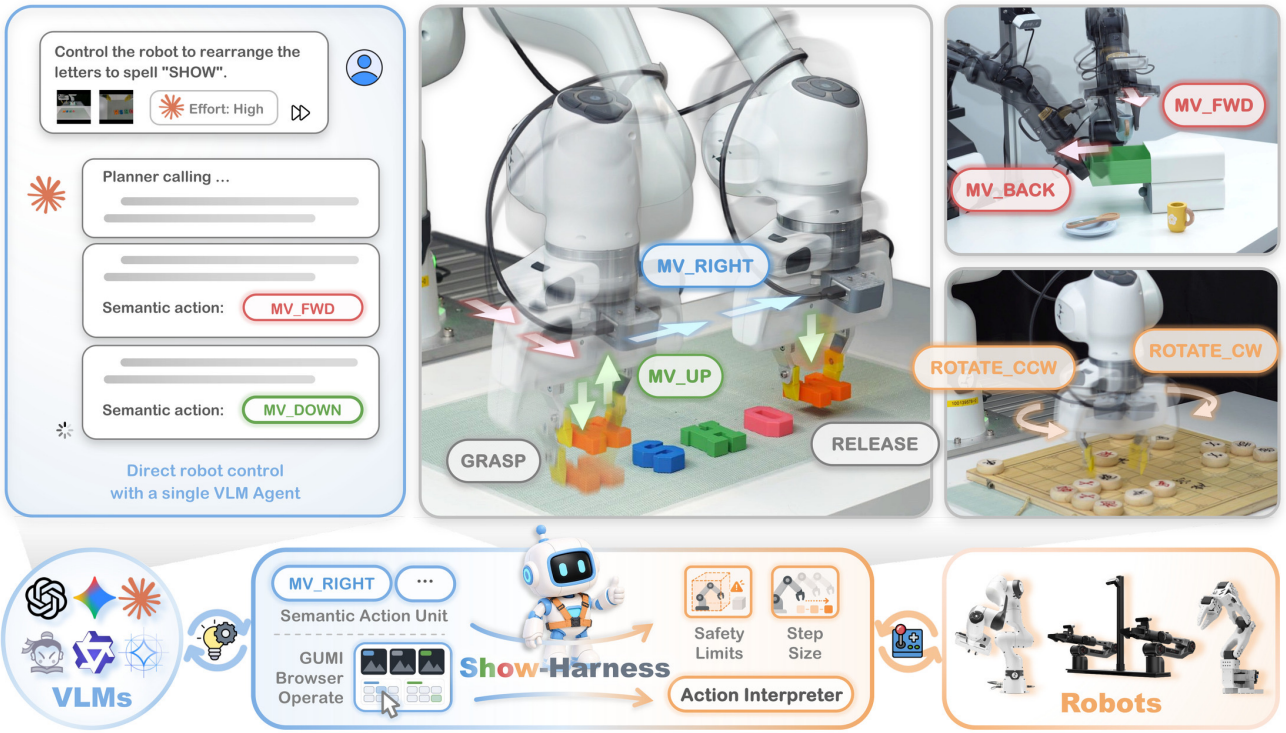
Show-Harness

- Now, let us see this idea on real robots. Here is our project demo video



Show-Harness

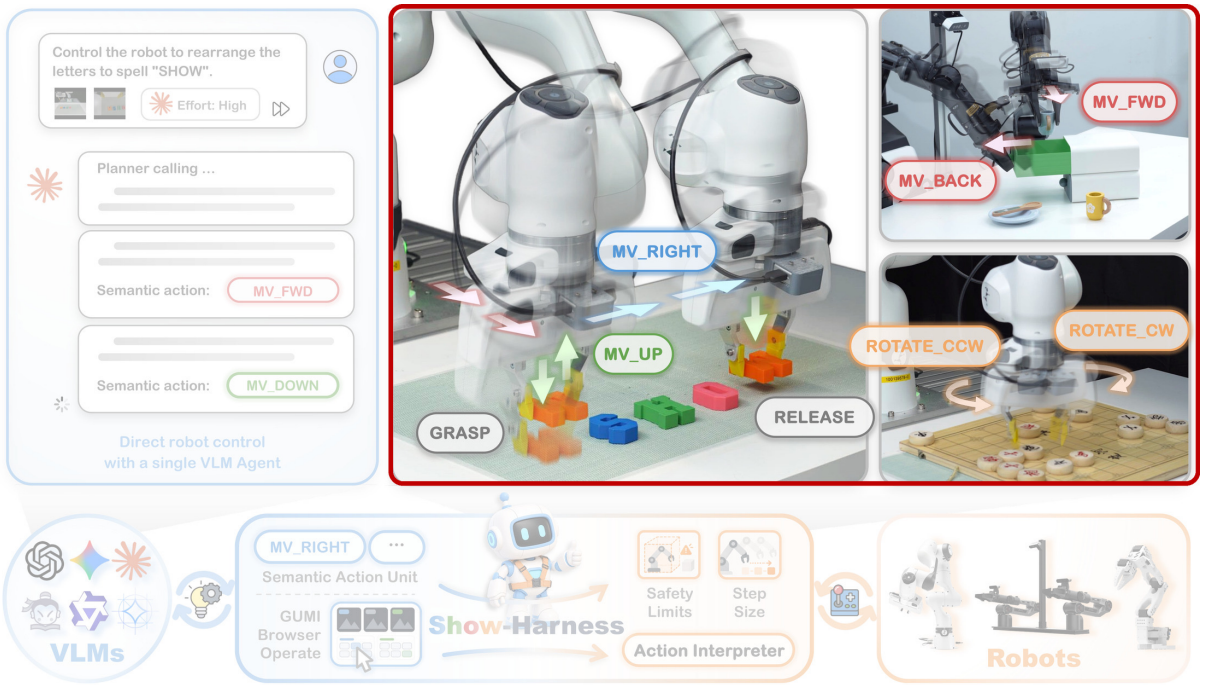
- Now let me explain how it works. Here is Show-Harness in one picture
- The VLM picks **one action word** at a time, like MV_RIGHT. An **interpreter** turns it into a motion
- The same words work for **different VLMs** and **different robots**



Yanzhe Chen*, Zechen Bai*, Zhijun Cao*, Wenzheng Zeng*, Kevin Qinghong Lin, Yiqi Lin, Guoqiang Liang, Kevin Yuchen Ma, Qiming Huang, Mike Zheng Shou (*equal contribution). arXiv 2609.10522

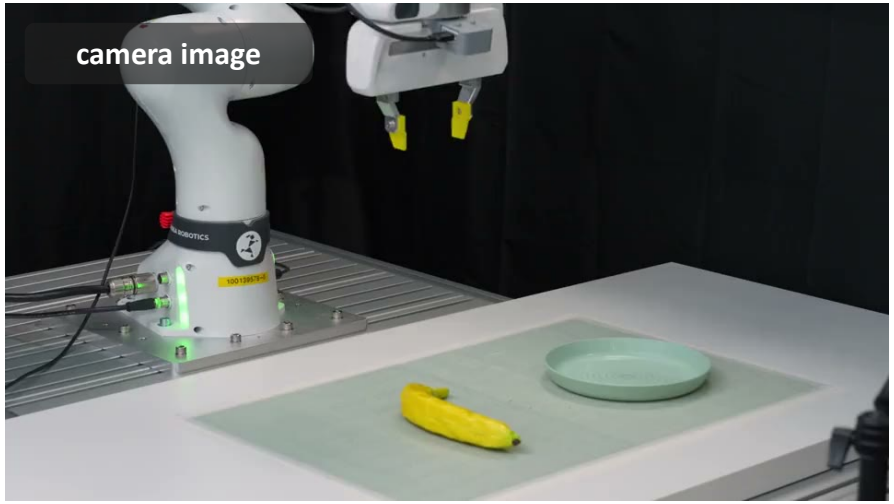
Show-Harness: Action Words

- We have action words to **move** in six directions, to **rotate**, to **grasp** and **release**, and **done**
- Each word is **one small step**, for example 2 centimeters or 15 degrees
- Directions follow the input **camera image**



Show-Harness: One Step

- Each step, the VLM reads the images, the task and **notes**, and answers with **one action word**
- Here, the **notes** help VLMs to **do reasoning**: gripper is 6 cm above the table → MV_DOWN




```
TASK: put the banana on the plate
STAGE: GRASP   TARGET: banana
Gripper: 6 cm above the table
Each step moves ~2 cm
Recent moves: MV_LEFT, MV_FWD
Choose one: MV_FWD, MV_BACK, ...
```

VLM

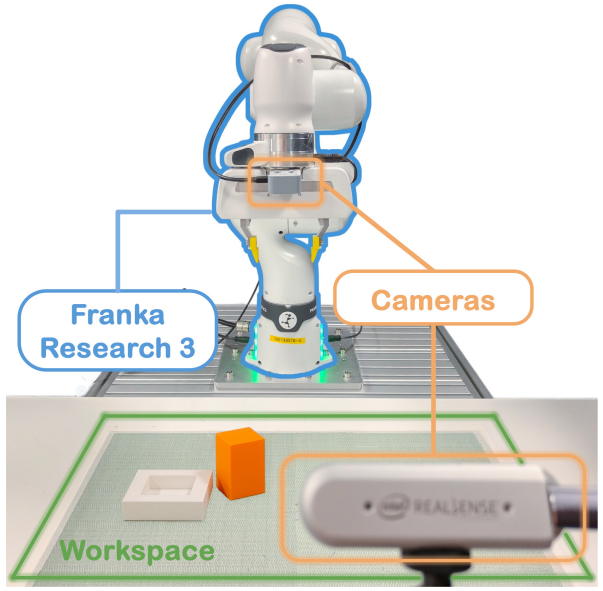
```
{"decision":
"MV_DOWN",
 "reasoning": "The
banana is between the
fingers."}
```

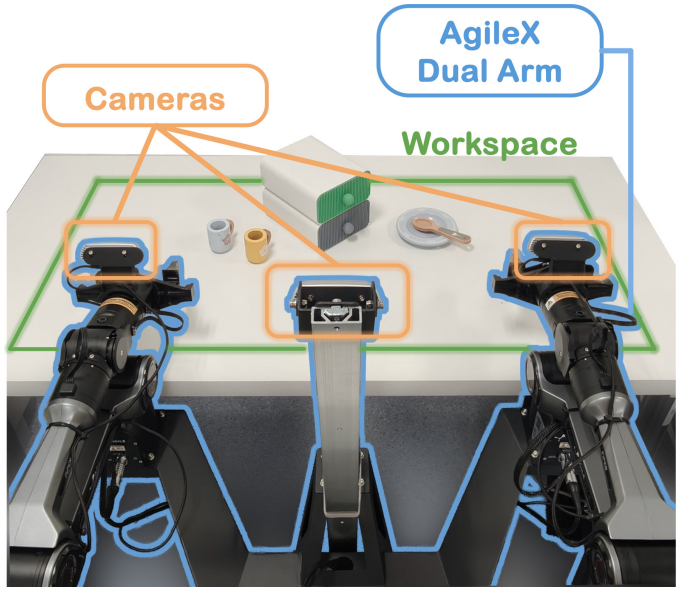
Then look again, and repeat

Prompt shortened from the paper's appendix

Show-Harness: Interpreter

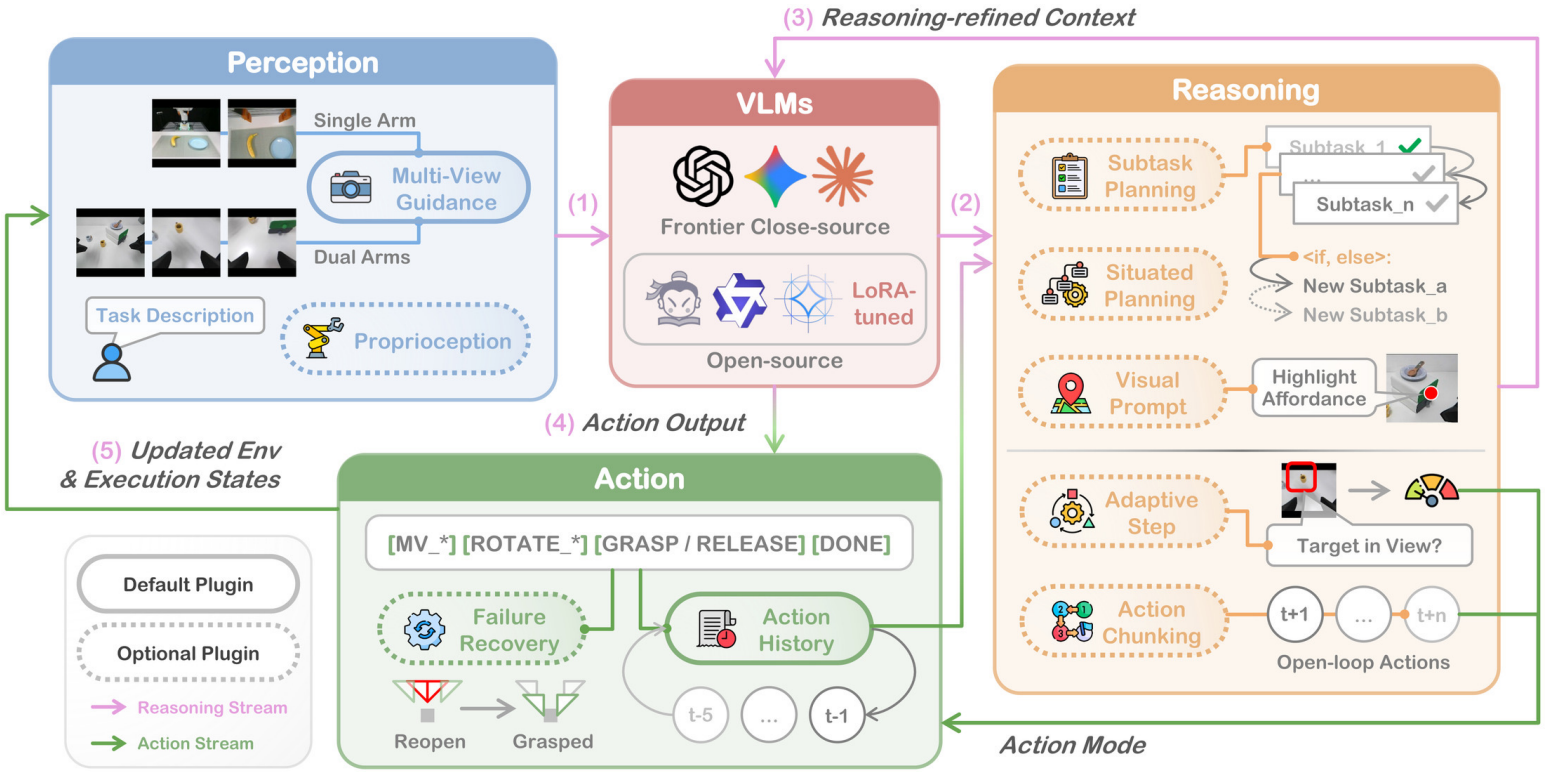
- The interpreter turns each word into motion: MV_RIGHT means **2 centimeters** to the right
- The interpreter is a fixed program, not a learned model. The robot's own control software still does the **low-level control**: it turns each joint motor by the right amount
- It checks **safety** first: the gripper must stay inside a safe area, and above the table. Unsafe moves are blocked
- A new robot only needs **a new interpreter**. Here are our two robots, Franka and AgileX





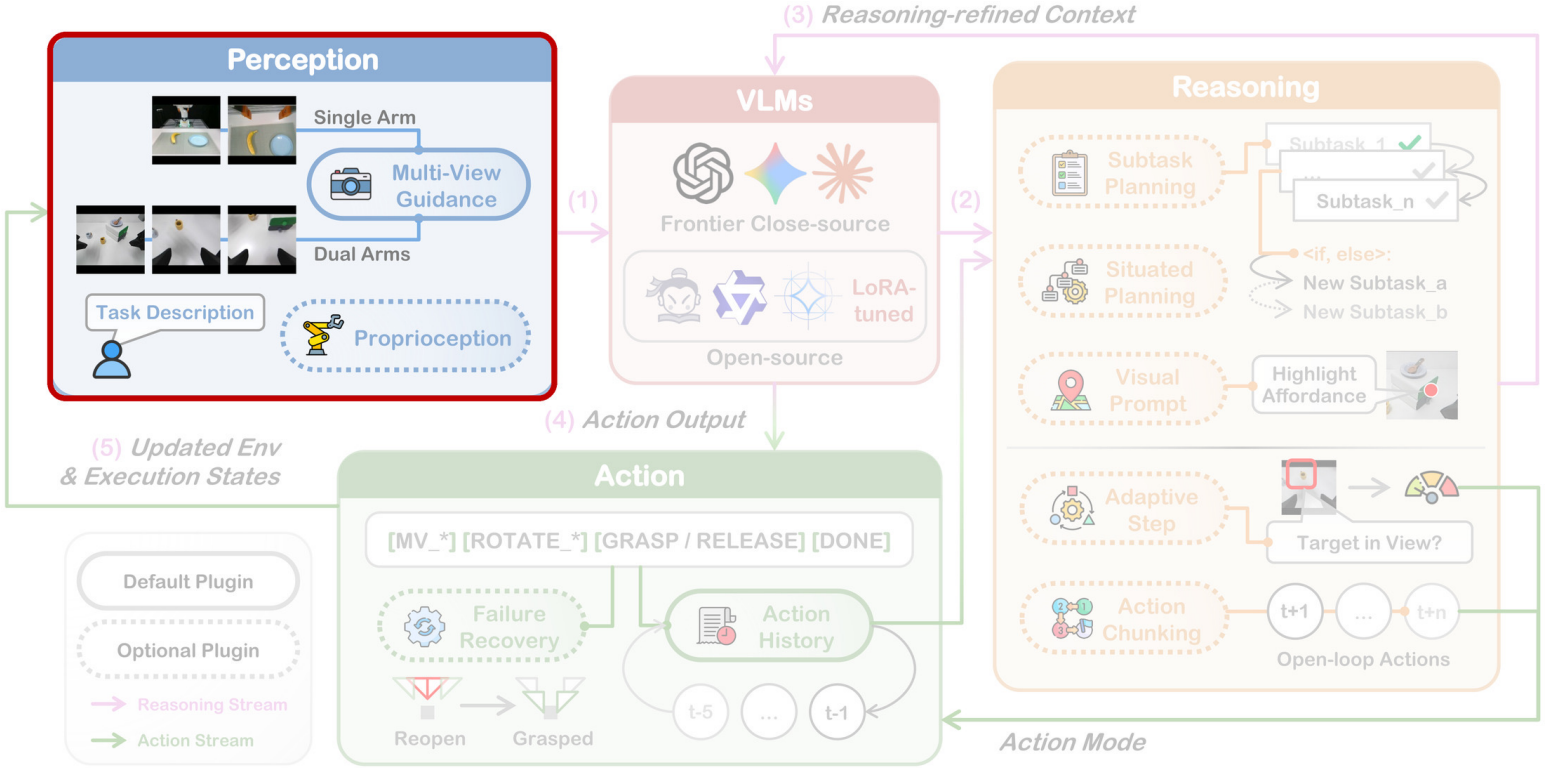
Show-Harness: The Loop

- Like Code2Video, Show-Harness has a **harness**: 3 stages & plugins that help the VLM
- Most plugins just add a short note to the text the VLM reads, like the gripper height. Each one can be turned on or off
- After every move, the cameras look again: a closed loop of **perceive, reason, act**



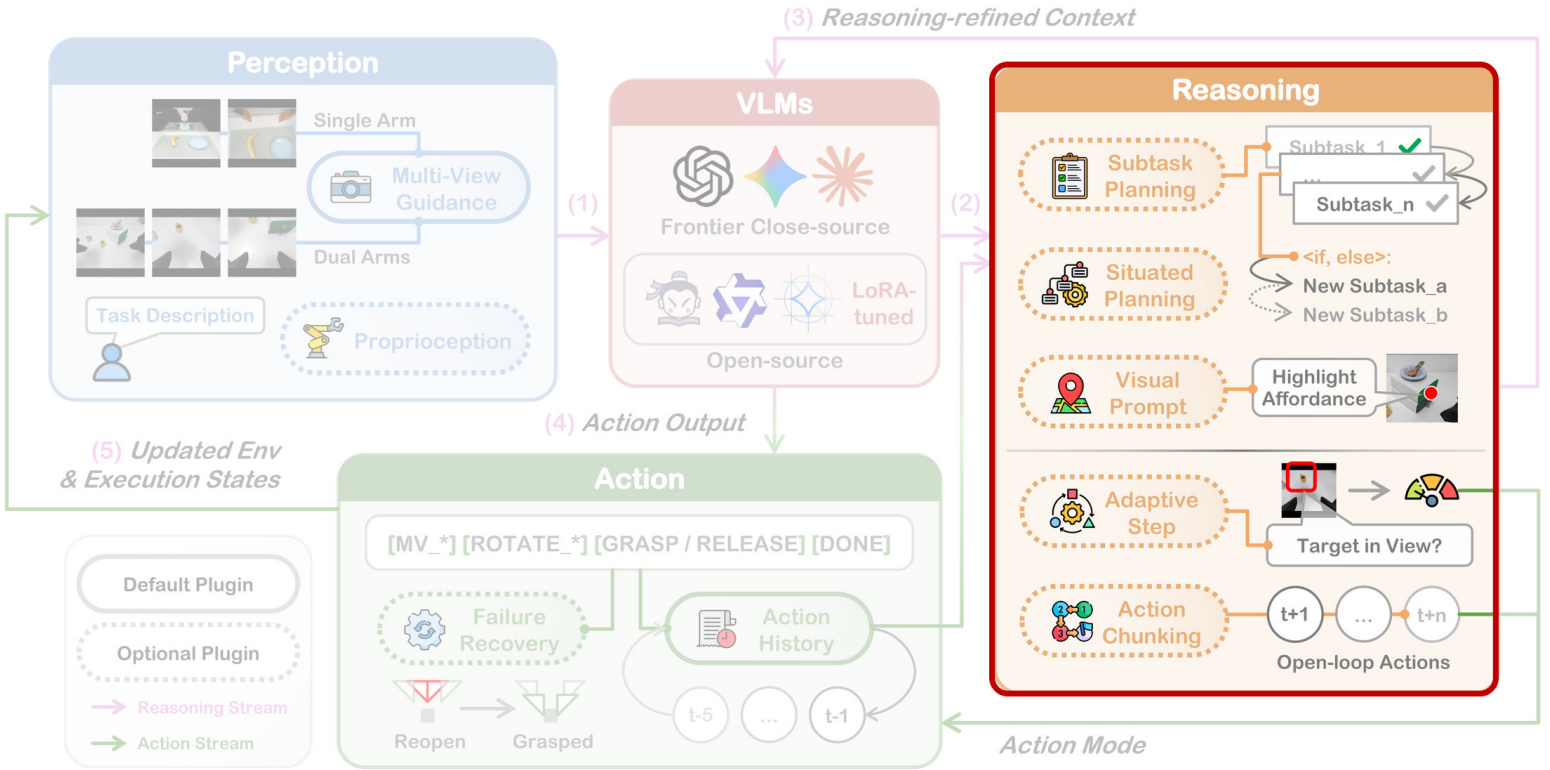
Harness: Perceive

- First, **perceive**
- **Multi-view guidance**: the front camera facing the table finds the object, and the wrist camera, next to the gripper, helps the fingers line up with it
- **Proprioception**: “The gripper is 6 centimeters above the table. The hand is empty.” Cameras do not show height well, so these words help



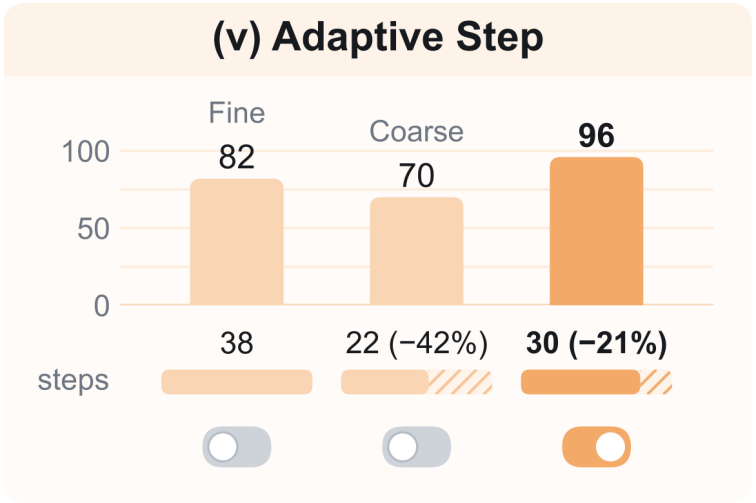
Harness: Reason

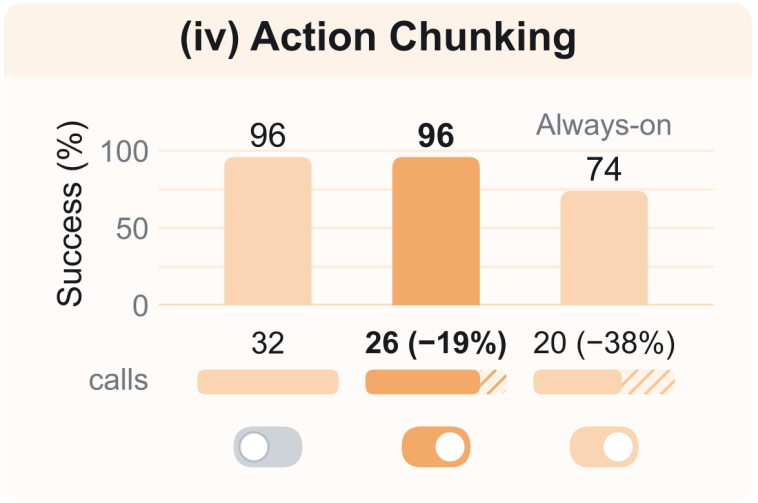
- Second, **reason**
- **Subtask planning** splits the task into stages, like grasp, lift and place
- Each stage ends with a check in the image, like “the banana is on the plate”
- **Situated planning**: if-else tasks, it looks first and decides later

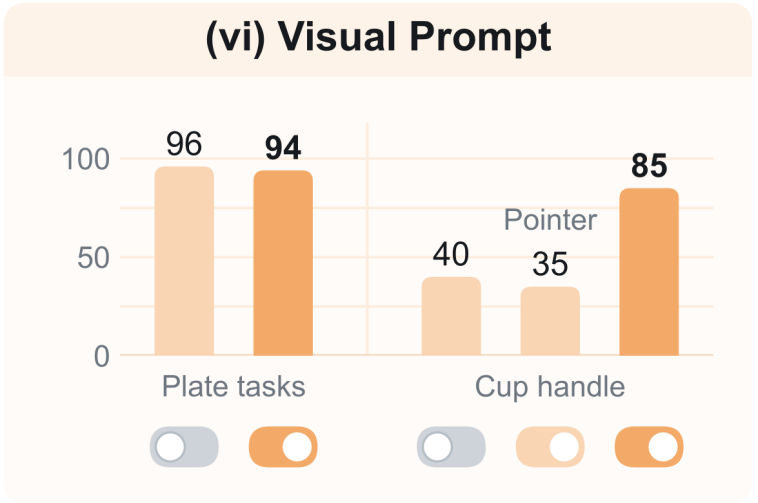


Harness: Three More Plugins

- **Adaptive step**: while the object is far, each word moves 4 centimeters. When the wrist camera sees it, each word moves 2 centimeters
- **Action chunking**: while the object is far, the model plans a few moves at once. The robot does them without asking the model again, which saves time
- **Visual prompt**: for correct affordance, like a cup handle, we ask a model to mark it on the image fristly. Then the model aims at the mark



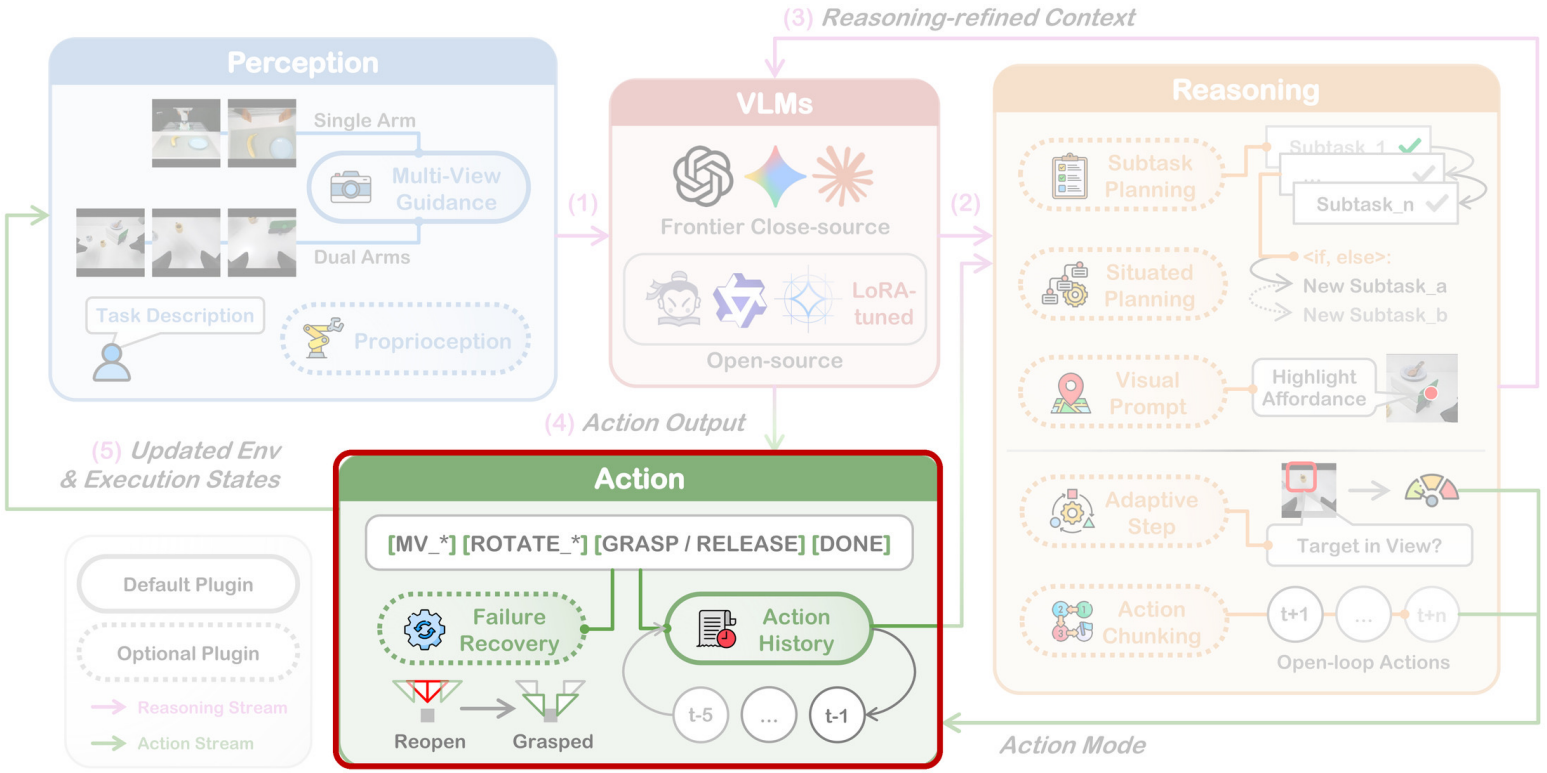




Our plugin tests on the real Franka (Figure 9 of the paper). Dark bar: our default

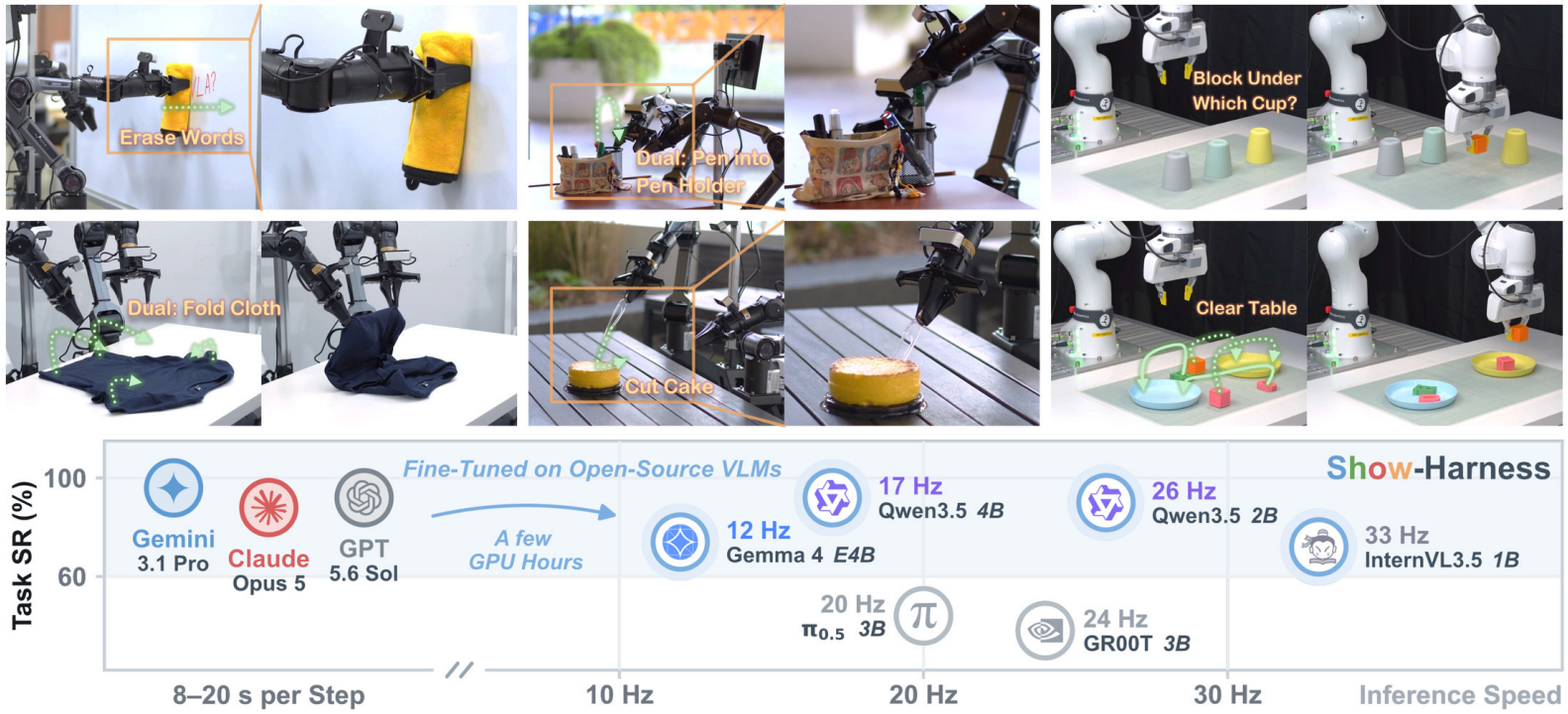
Harness: Act

- Third, **act**
- **Action history** keeps the last 5 moves, so the model does not go back and forth
- **Failure recovery**: if the gripper closes on nothing, the harness notices. It opens the gripper and goes back to the grasp stage



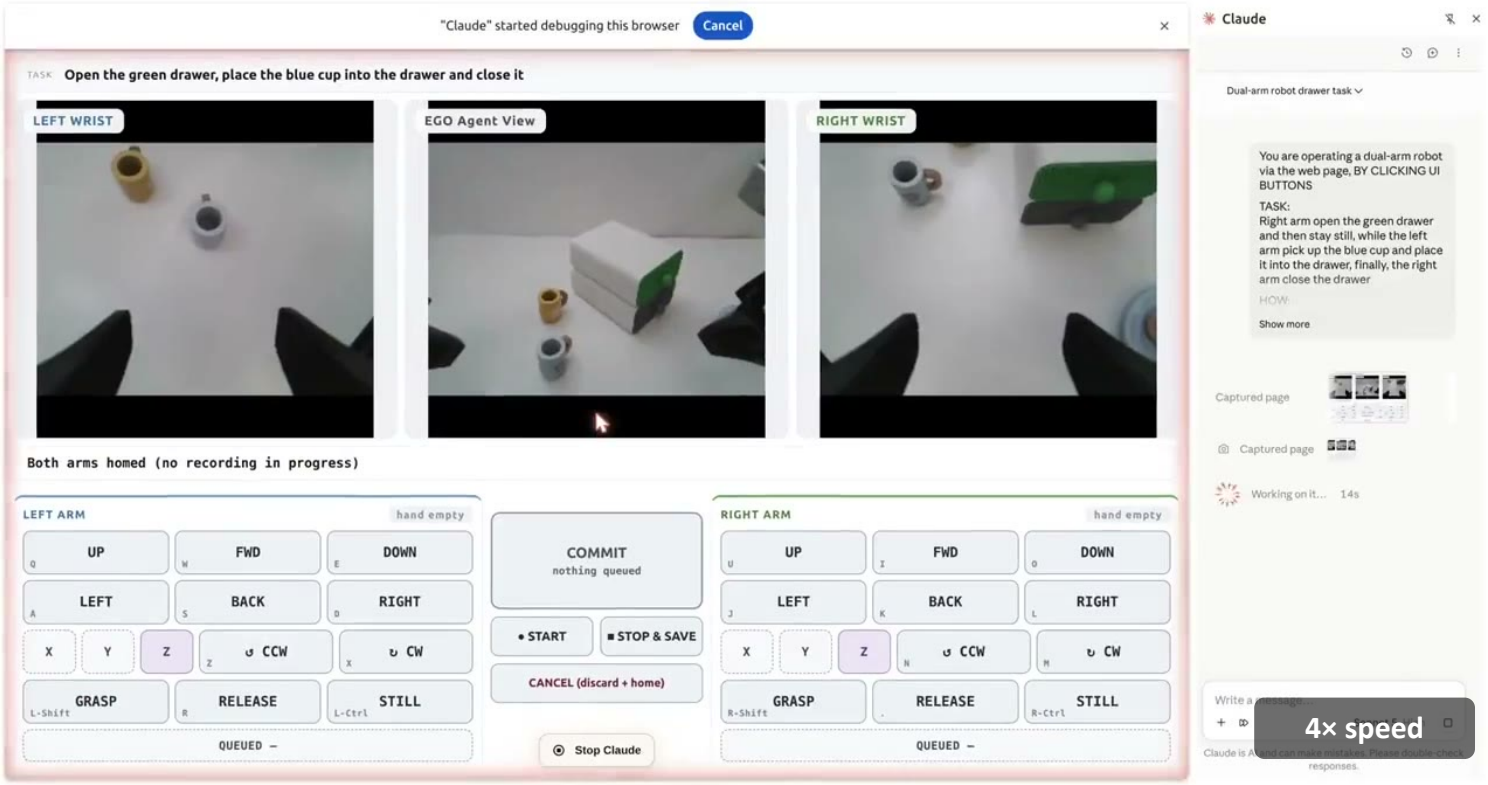
Show-Harness: Two Modes

- The same actions words can also be used to train a small model
- **Zero-shot**: a frontier model, like Gemini, GPT or Claude, runs the robot **directly**
- **Fine-tuned**: we take a small open-sourced VLM, and train it on our robot data
- It has 2B parameters, which learns in **under 2 hours**, and it also runs much faster



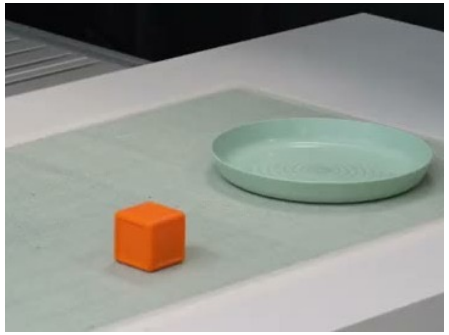
GUMI: Play Robots in a Browser

- We built **GUMI (GUI Manipulation Interface)** : every action word is a button
- People can play the robot from a laptop, and **AI agents** can click the same buttons
- In this video, **Claude in Chrome** opens a drawer with two arms
- Each click is saved with the images: **164 real single-arm runs**, with no special hardware

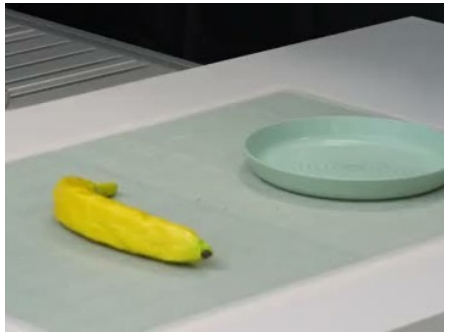


Experiments: Setup

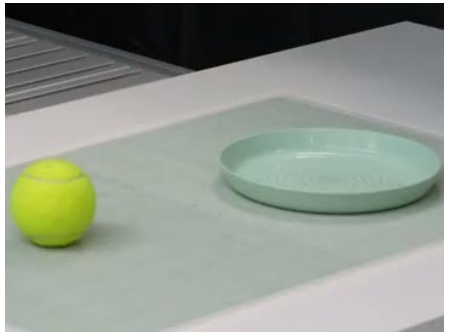
- We use two real robots: a **Franka** with one arm, and an **AgileX** with two arms
- We have 10 tasks: 5 objects, each to a plate or a bowl. Two objects are **new**: they never appear in our training data
- We compare with two strong VLAs, π0.5 and GR00T, fine-tuned on the same 164 demos
- We also compare with four agents: two of them plan and let a VLA move the arm, and two write robot code



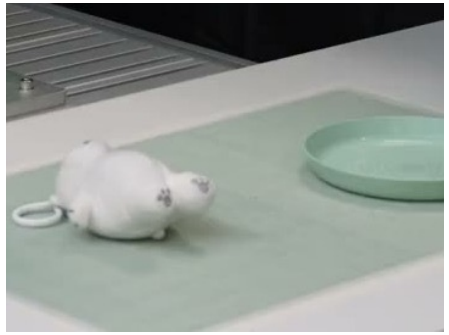
block



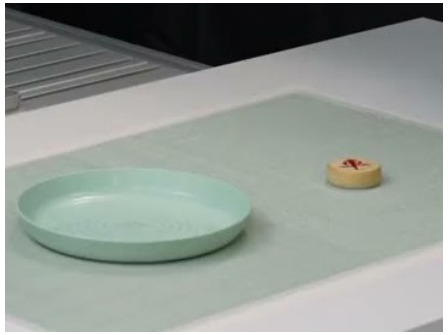
banana



tennis ball



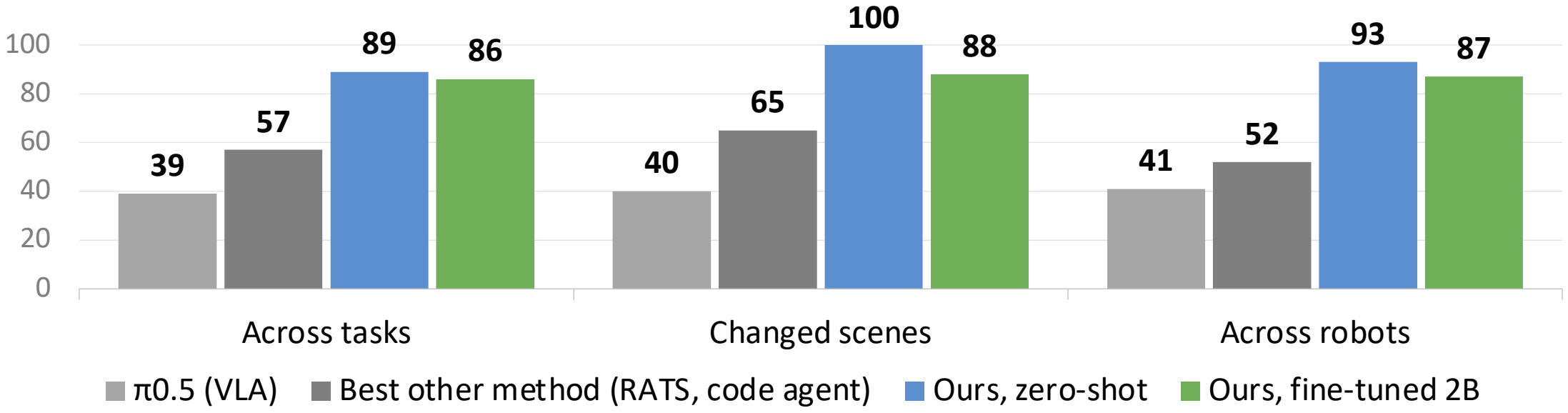
teddy bear (unseen)



chess piece (unseen)

Results: At a Glance

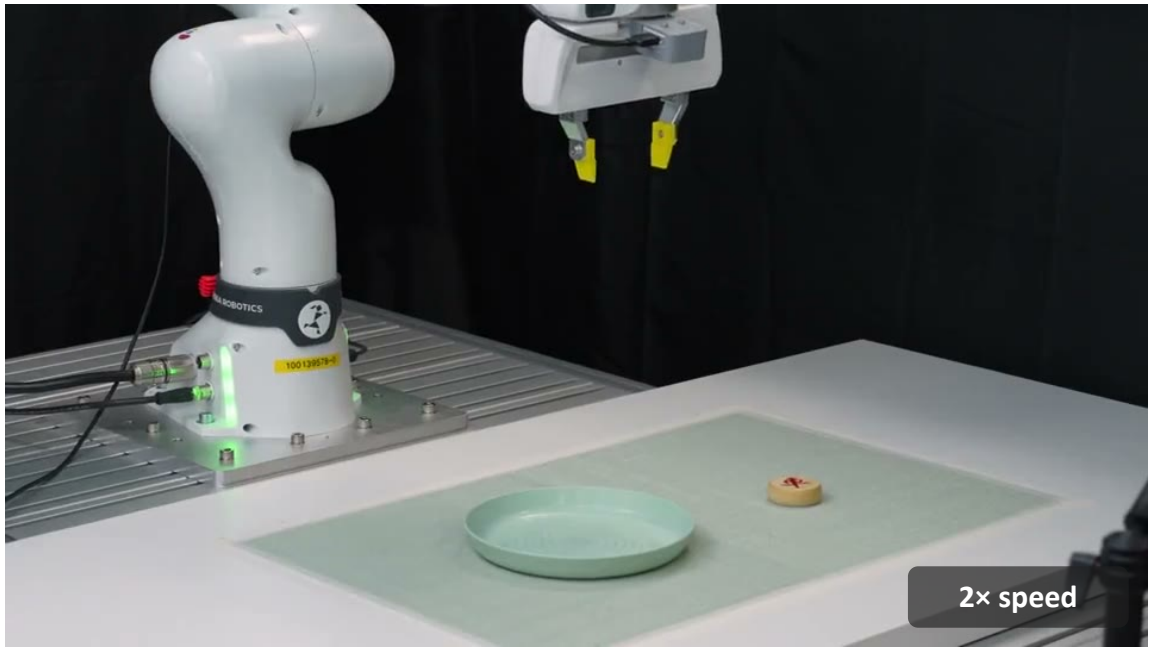
- Here are the success rates on real robots, with 10 tries per task. All trained models use **the same small data**
- Gray bars are the baselines we compare with, blue is our zero-shot agent with Gemini 3.1 Pro, and green is our fine-tuned model with 2B parameters (Qwen3.5-2B)
- Both of our modes are much higher in all three groups. Here are some videos



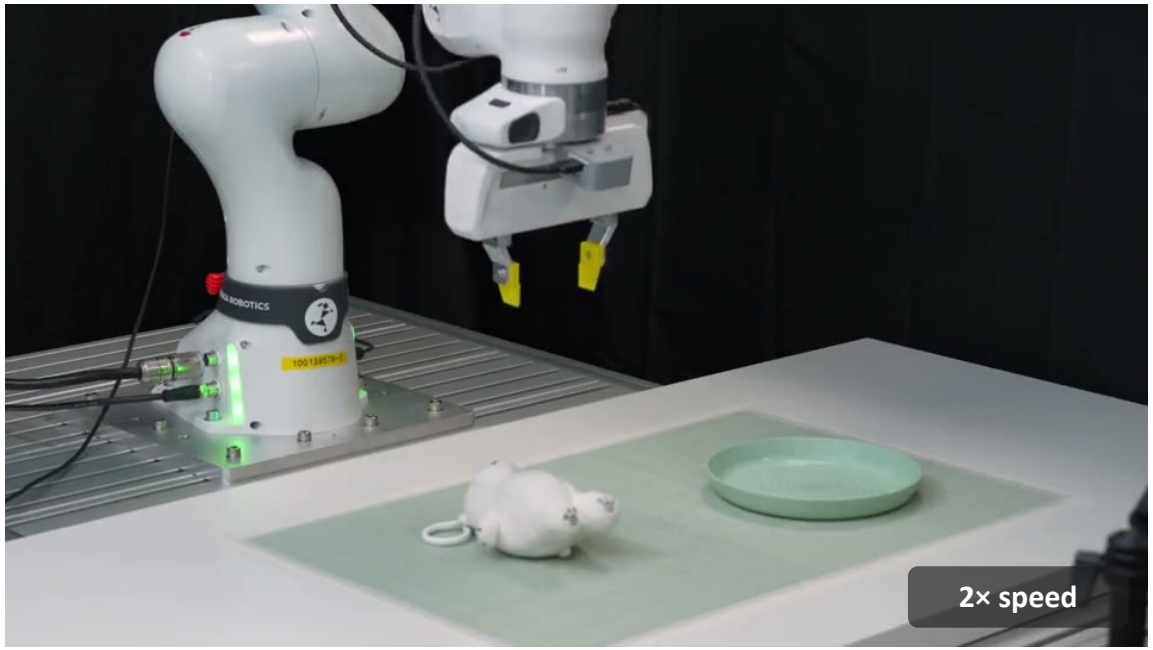
Changed scenes: background, lighting, viewpoint, distractors (+ sim-to-real for trained models). Table 2 of the paper

Results: Across Tasks

- Across the 10 tasks, zero-shot gets **89%**, fine-tuned **86%**, and the best other method 57%
- On the small, unseen chess piece, zero-shot gets **10 out of 10**, fine-tuned **9**, and π0.5 only 1



chess piece → plate (unseen)



teddy bear → plate (unseen)

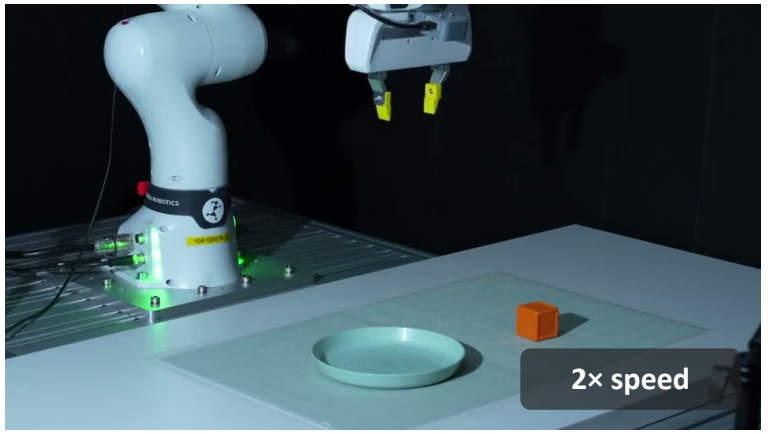
Results: Changed Scenes

- Next, we change the background, the lighting and the camera view, and we add extra objects
- Zero-shot gets **20 out of 20** in every case. Fine-tuned gets **18 or 19**. π0.5 gets 9 to 11



new background

ours 20/20 · π0.5 11/20



new lighting

ours 20/20 · π0.5 9/20



extra objects

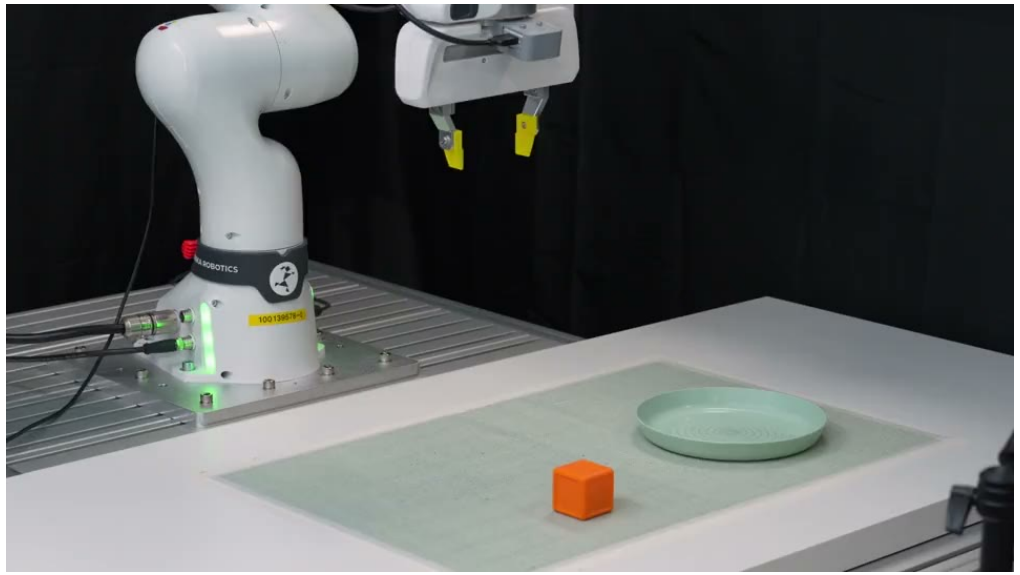
ours 20/20 · π0.5 10/20

Results: From Simulation to Real

- Now a harder test. We train **only in simulation**, with 230 runs
- A simulator is like a video game of the robot. Data there is cheap, but the images look different from the real lab, also physical rules (**Sim2Real Gap**)
- Then we test on the real Franka



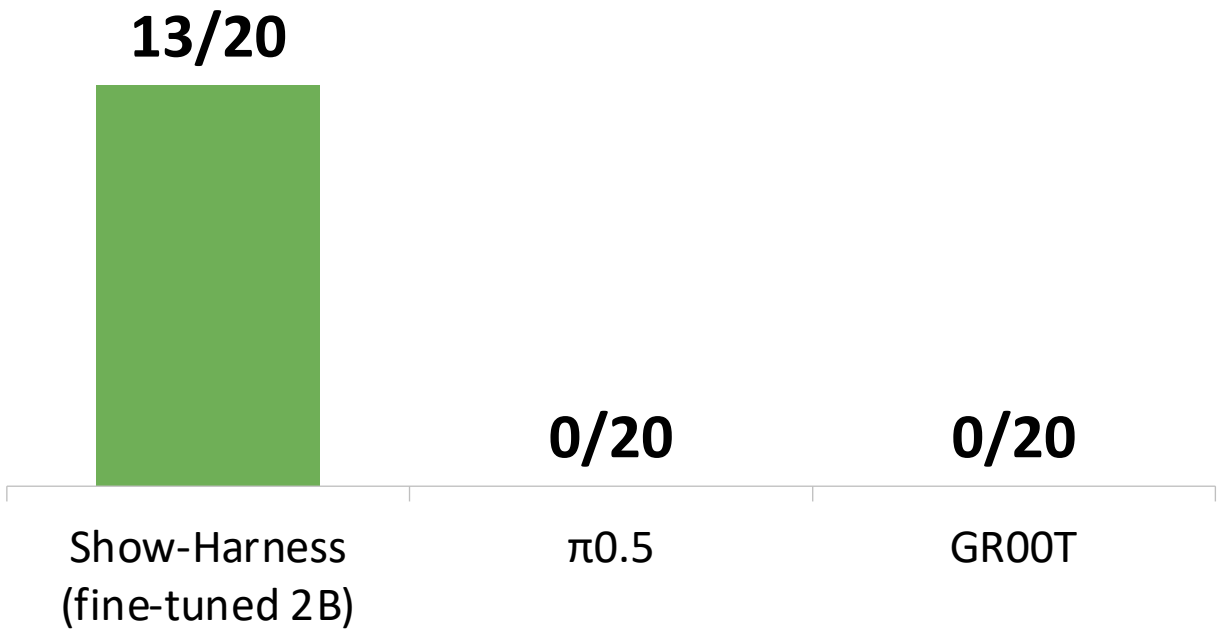
Train: Simulation

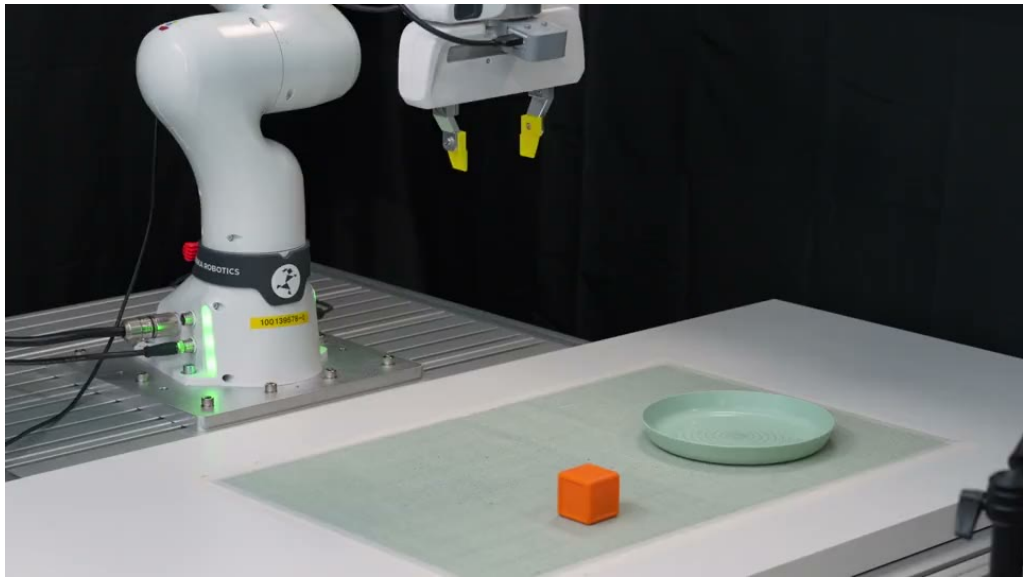


Test: the real Franka

From Simulation to Real: The Result

- Our small model succeeds **13 times out of 20**. π0.5 and GR00T succeed **0 times**
- Our guess: MV_LEFT means the same 2 centimeters in simulation and in the real lab
- So the model only learns which word to choose, easier with limited data budget





the test: the real Franka

Results: Across Robots and Two Arms

- We keep the same VLM and action words on the new robot, and only change the interpreter: **48 out of 50** on Franka, **45** on AgileX
- With two arms, one VLM decides for both arms. If one arm should wait, it says STILL
- Passing the banana works **70%** of the time
- Two separate agents (each controls one arm) get 30%



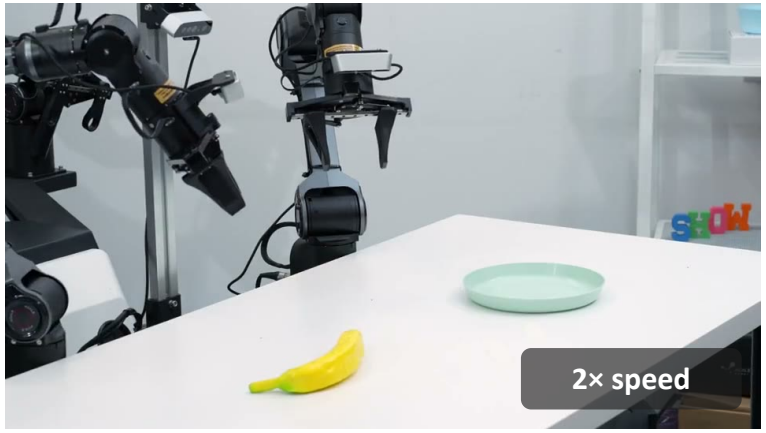
Franka (7 joints)

zero-shot: 48/50



AgileX (6 joints)

zero-shot: 45/50

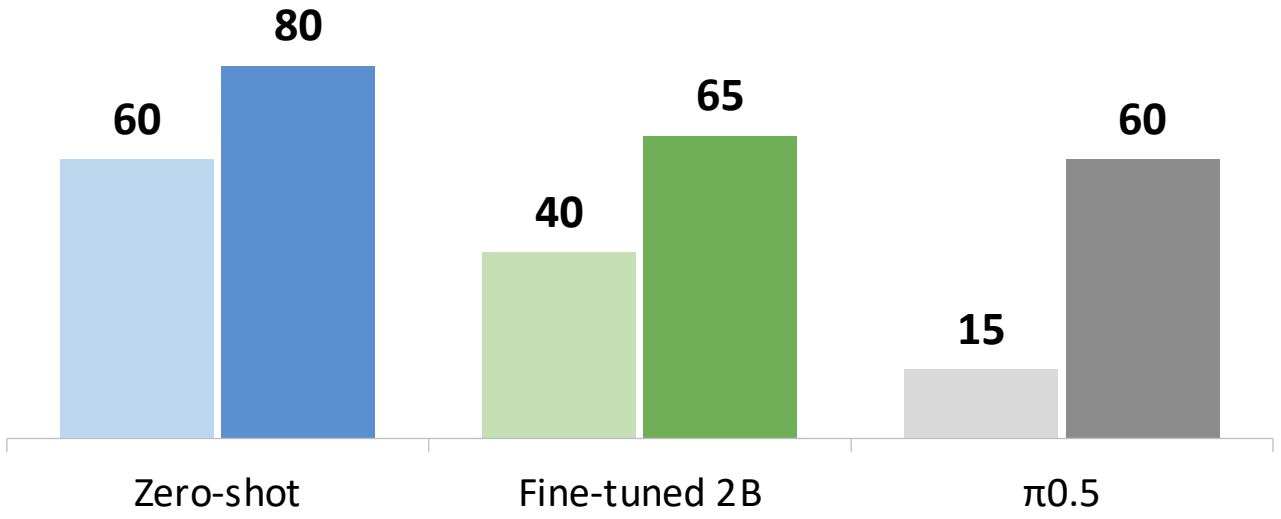


AgileX: pass the banana

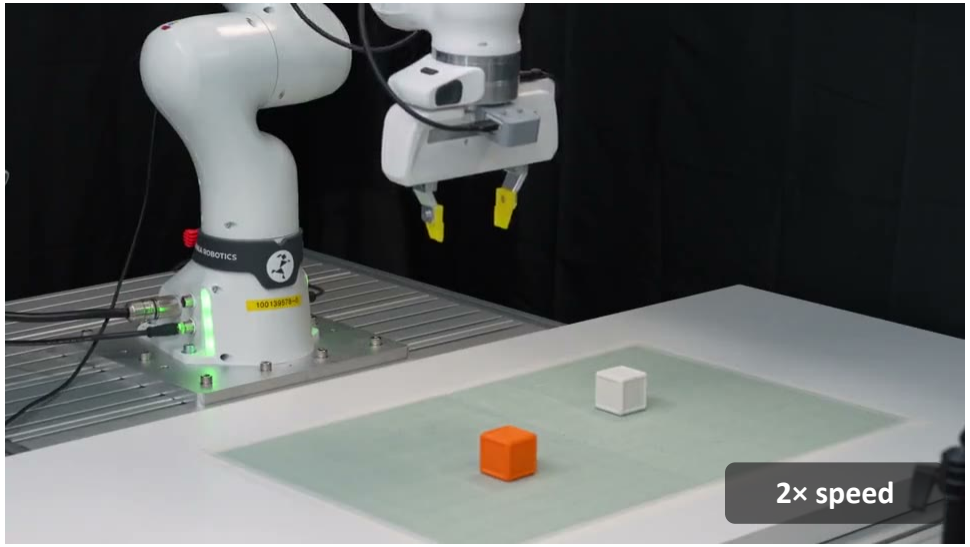
one agent: 70%

Results: Finer Moves

- For more fine-grained tasks like stacking blocks and inserting a peg, we only change the interpreter: each word now moves **1 centimeter instead of 2**
- Zero-shot goes from 60% to **about 80%**, and fine-tuned from 40% to **65%**, **with no retraining**
- π0.5 gets only 15% with the same data, and it needs **extra demos of these precise tasks** to reach about 60%



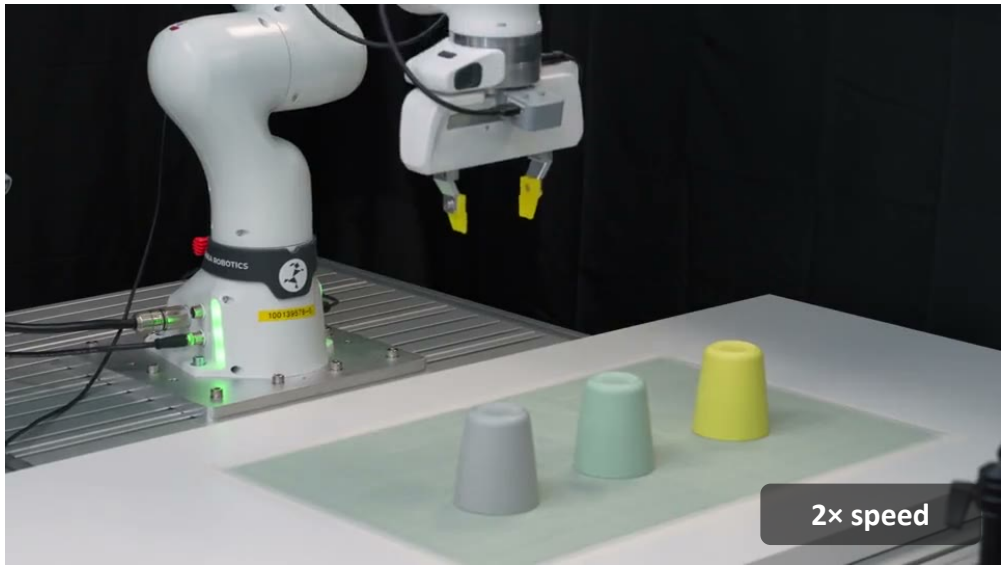
light bar: 2 cm steps or default data; dark bar: 1 cm steps or fine-grained data



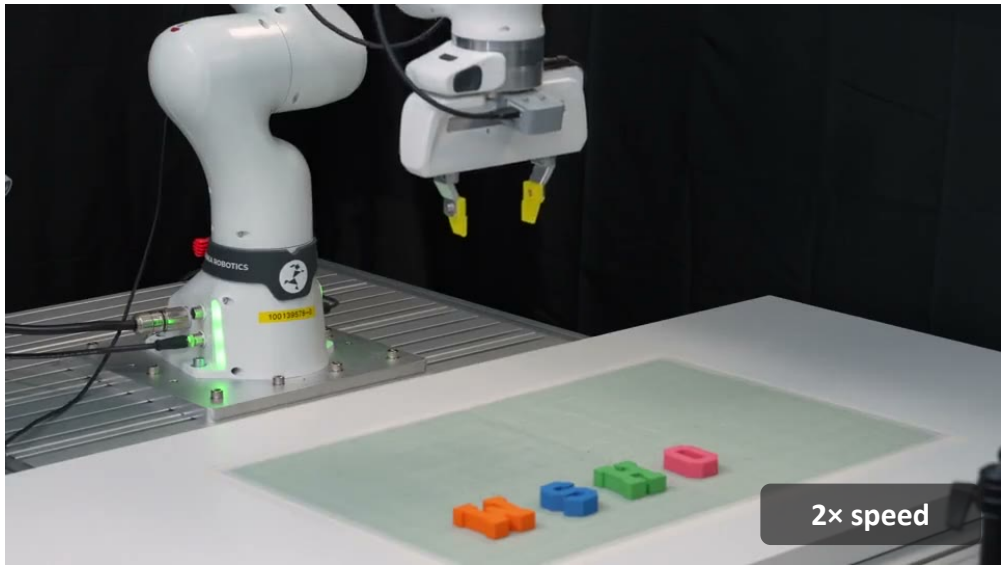
block stacking

Results: Think, Then Act

- Some tasks need thinking: find the block under one of three cups, or spell “SHOW”
- Zero-shot with **situated planning** gets **85%**. π0.5 gets **0%**, and 5% with Gemini’s plan
- Our small model gets 10% alone, but **70%** with Gemini’s plan



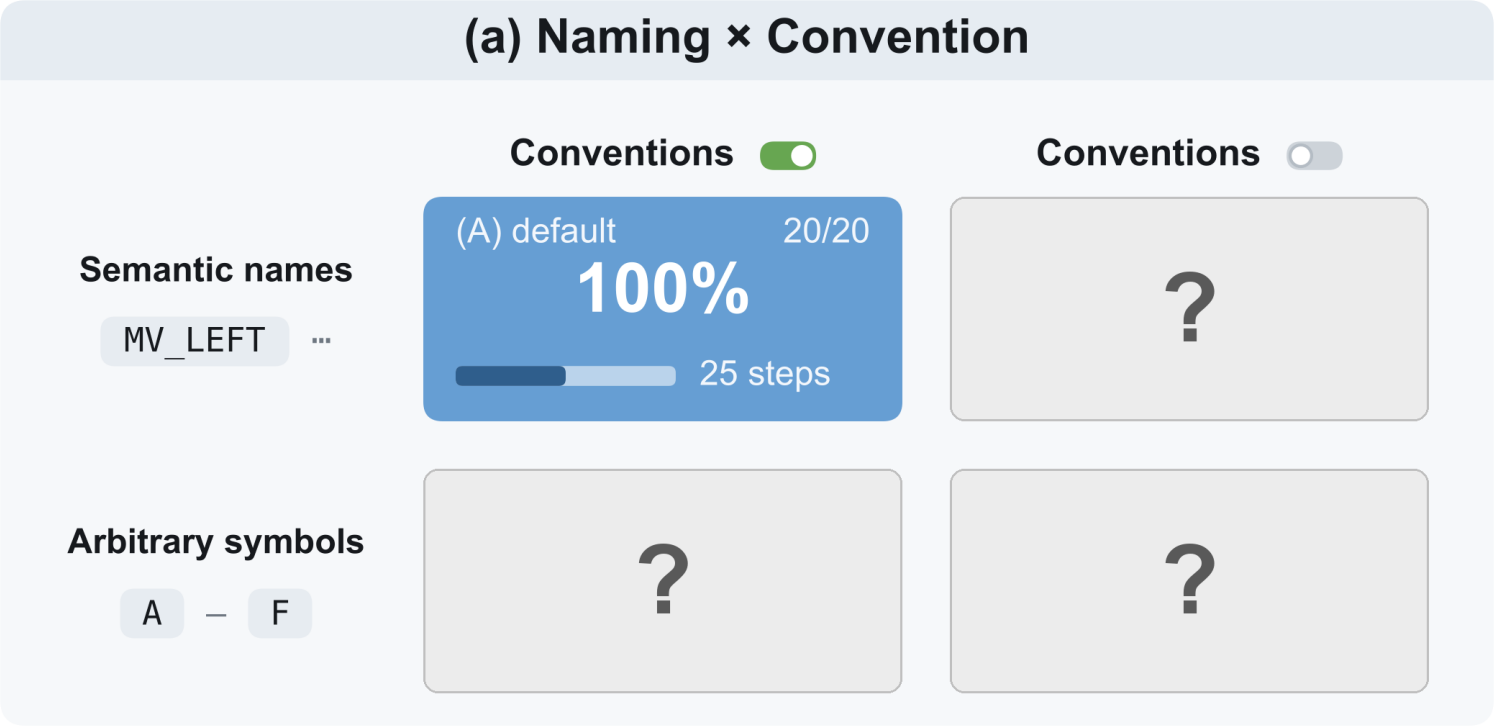
which cup?



spell SHOW

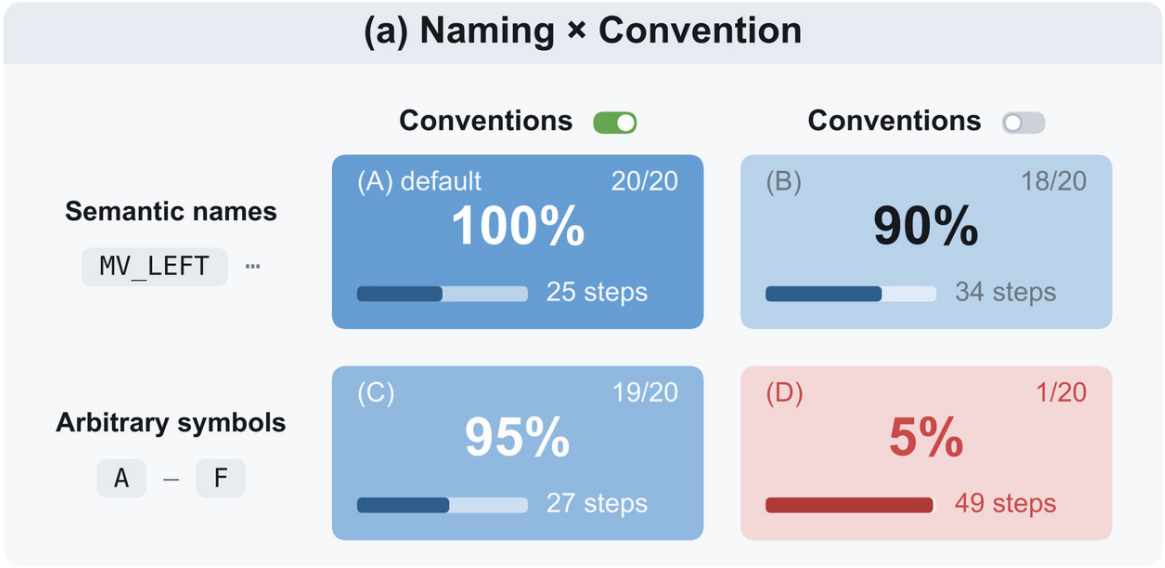
Quiz: Why Do the Words Work?

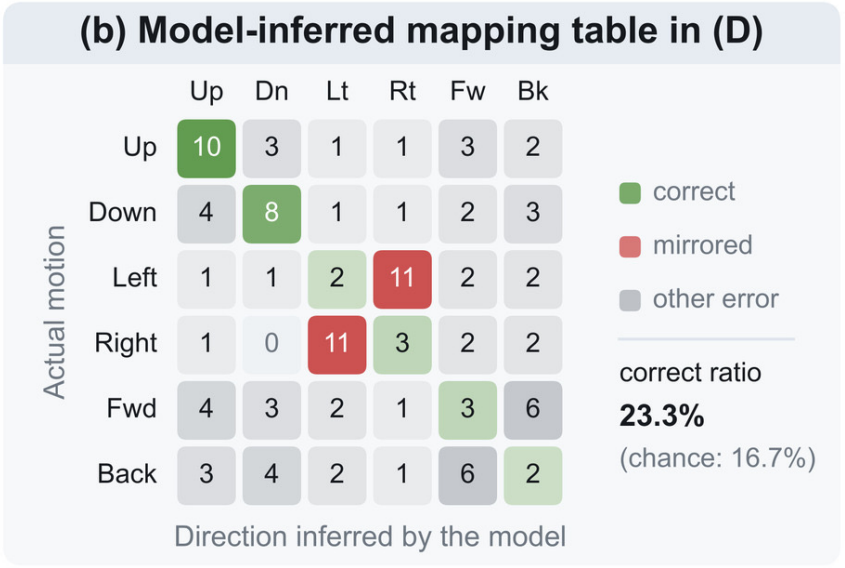
- The results look good. But why do the action words work well? Let us do an ablation
- We **rename the buttons**: MV_LEFT → “A”, MV_RIGHT → “B”, …
- We **give a rule sheet**, like “A moves left”, or **no rules**, so the model must find out by trying
- Please guess: with no rules, does it succeed more than half of the time?



Quiz: Answer

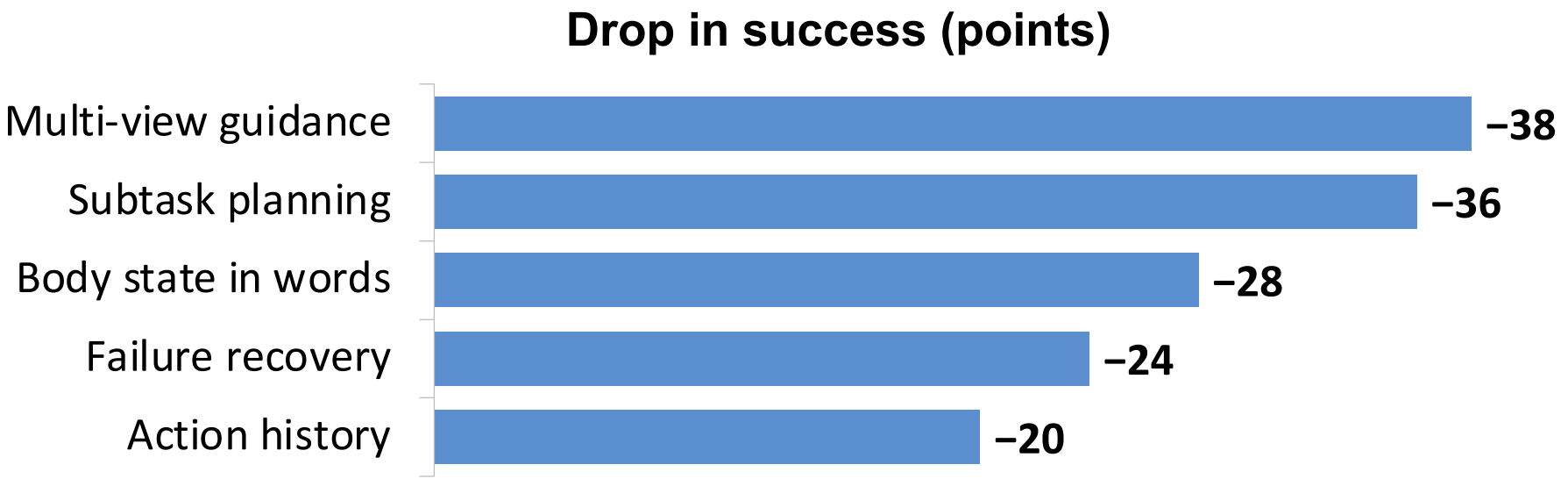
- Our default setting, names with rules, gives 100%
- Names alone give 90%, and symbols with rules give 95%
- But symbols with no rules give only **5%**. By trying, it learns only **23%** of the directions; guessing gives 17%
- So the model must be told **what each word means**, **by a clear name or a rule**
- Today, it still **cannot find this out** by dicovering





Analysis: Which Plugin Matters?

- So the words matter. What about the harness? We remove one plugin at a time (ablation)
- On the five plate tasks, the full system gets 96%.
- Without multi-view guidance, it falls to **58%**, the biggest drop
- Action chunking keeps 96%, and it **asks the model 26 times instead of 32**
- On a cup handle and a hidden block, the visual prompt and situated planning lift success from about 40% to **85%**



2026: Frontier Models Meet Robots

- That was our work. Since April, many groups let frontier models use robots, each with its own interface and harness
- An interface is how the model gives orders to the robot, like words, numbers or code

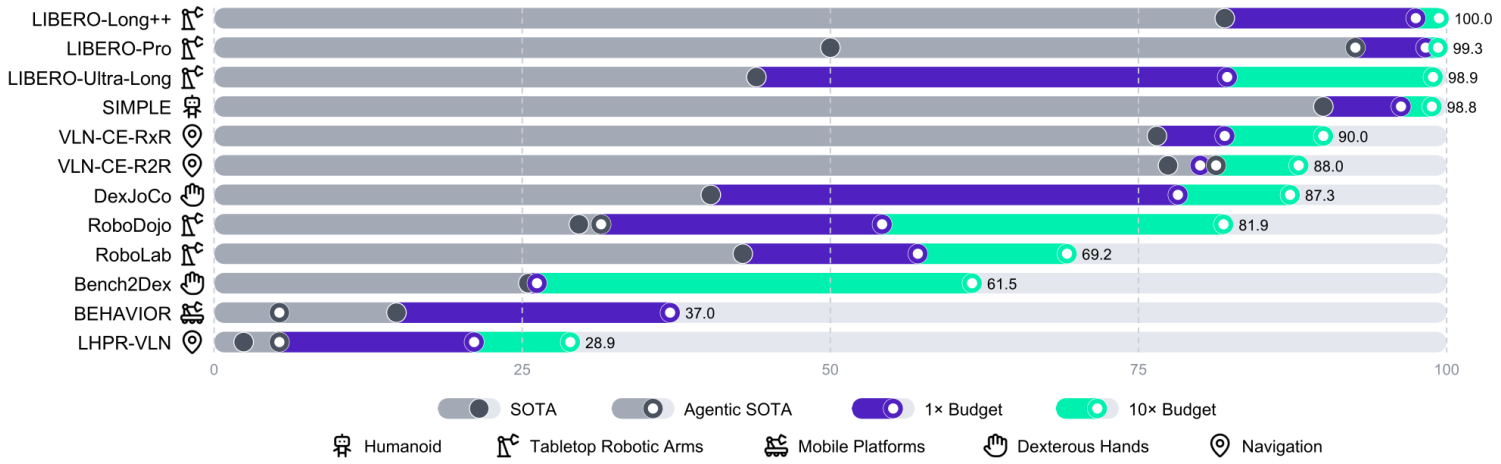
June 16	Guava	a whole harness, taught to one small open model
July 9	**Anthropic: Claude plays robotics**	the interface matters as much as the model
July 30	**Gemini Robotics 2**	a reasoning model is the “high-level brain” over a VLA
September 3	**GPT-6 Astra**	a general model; many teams let it control robots directly
September 9	**Show-Harness (ours)**	small action words; zero-shot and a small model
September 11	Agent as Policy	an agent writes robot code: at least 80% in 7 of 8 setups
October 15	SenseNova and others	12 benchmarks and 8 real robots
October 18	RoboCurve and others	the best models’ robot scores double about every 54 days
October 19	Embodied First Exam	about 1,100 robot tasks for agents that write code

- Next, let me show the two big tests from October in more detail

arXiv 2606.18363; anthropic.com/research/claude-plays-robotics; deepmind.google (Gemini Robotics 2); arXiv 2609.12541; arXiv 2610.22471; arXiv 2610.25287; embodied-first-exam.ai

Has GPT-6-Astra Achieved Embodied Intelligence?

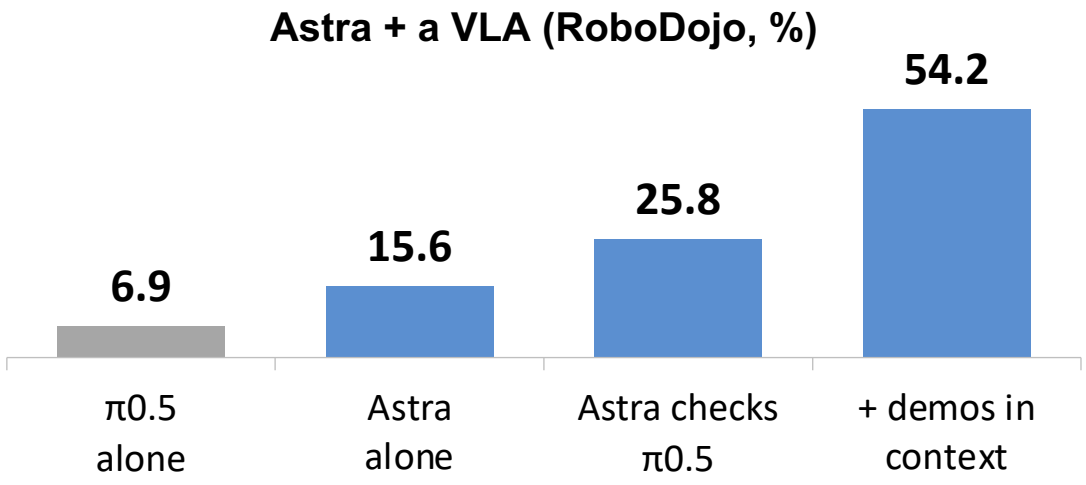
- The first big test is from SenseNova and a joint team. They tested GPT-6 Astra on 12 benchmarks, and on 8 real robots
- When a few example runs are shown in its prompt, it matches or beats the best published results on most of them
- But one easy run takes **nearly an hour and about one million tokens**. 😱
- Their answer: Astra **has real robot skills**, but it is **not practical yet**

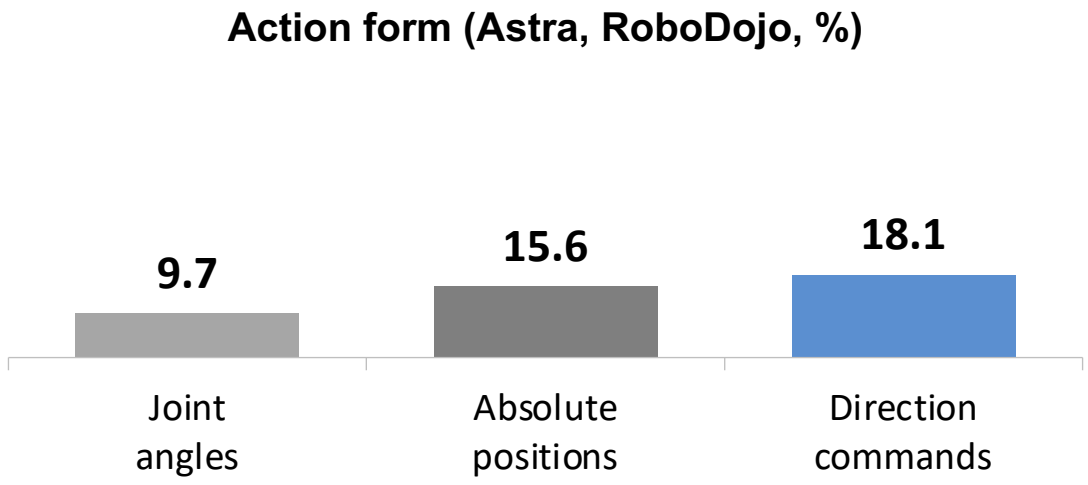


arXiv 2610.22471, Fig. 1. Gray dots: best published results. Purple and green: Astra (normal and 10× step budget)

What They Found About the Harness

- When Astra checks a VLA's actions, success goes from 7% to 26%
- Direction words like move left do a bit better than exact positions, and exact angles for each joint do the worst
- Their advice: let the model choose **meaningful action words**, and let the robot's control software do the **precise motion**. This is the **same idea as Show-Harness**

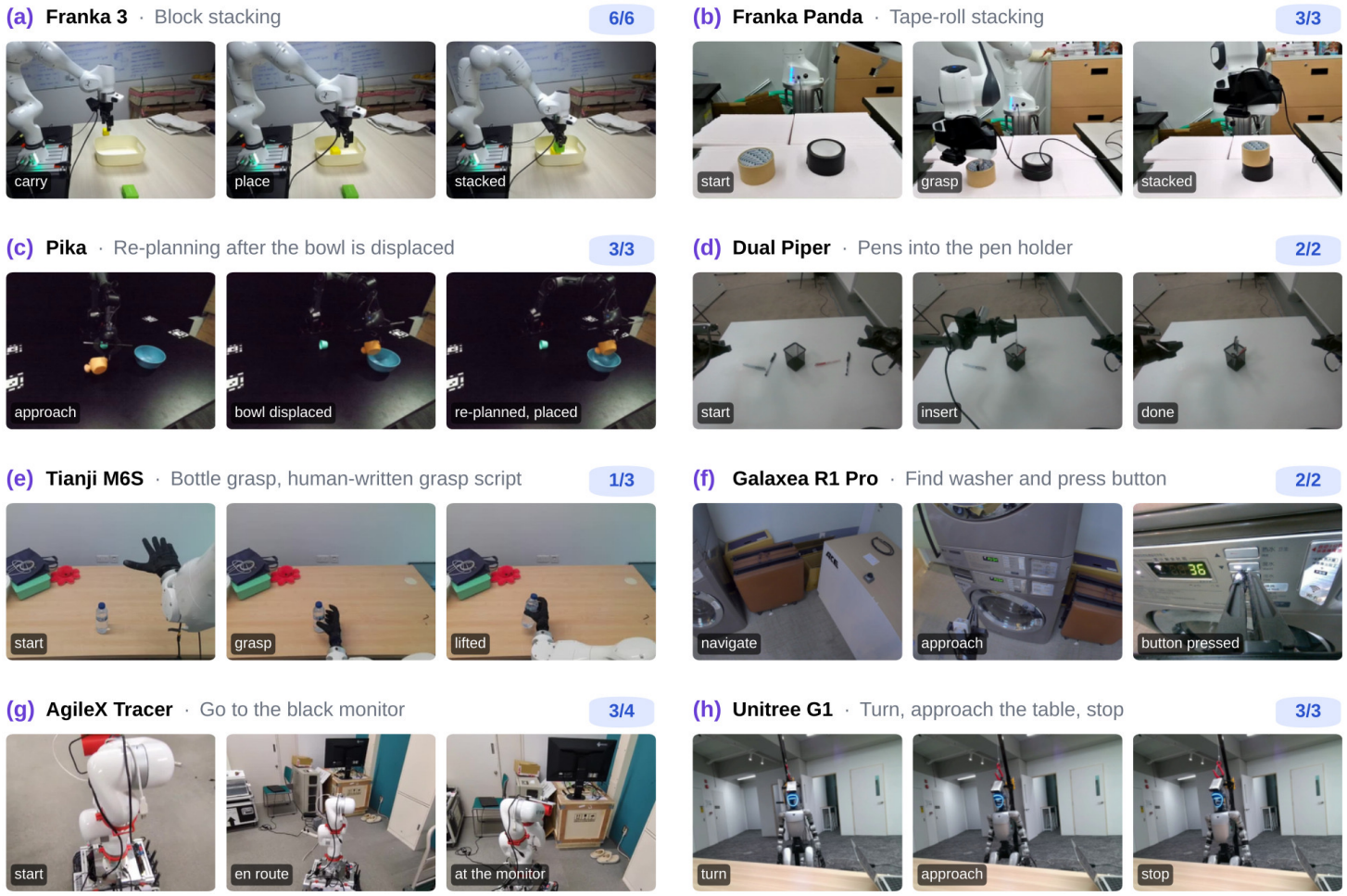




"Has GPT-6-Astra Achieved Embodied Intelligence?", arXiv 2610.22471, Sec. 3 and Fig. 7 (simulation)

The Real-World Gap

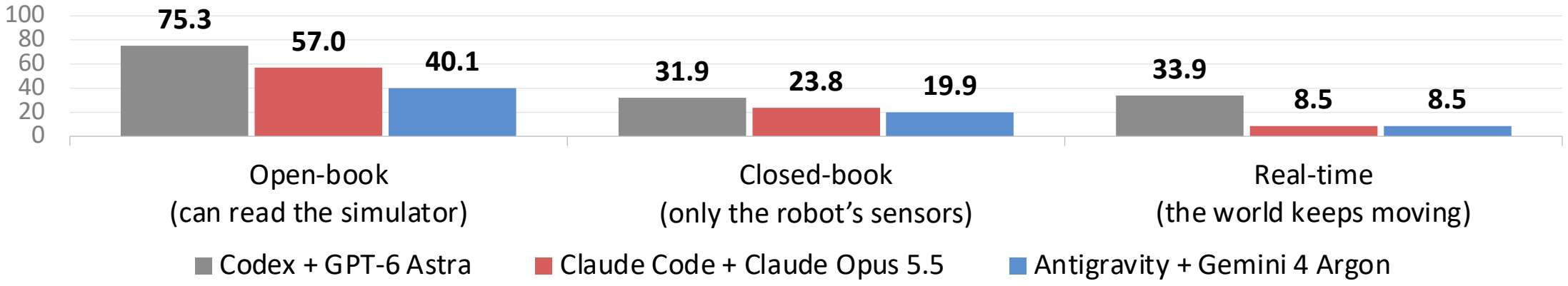
- They tested 8 real robots
- Pick and place: **17 out of 18**
- Plug insertion: **0 out of 11**
- On the human-shaped robot, the model spends **90%** of its time on thinking
- They suggest a new test: **the world keeps moving while the model is thinking**
- Our results agree: exact placing is still hard



Successful runs shown; plug insertion (0/11) is not pictured

Embodied First Exam: A Test for Coding Agents

- The second big test is **Embodied First Exam**, released last week
- It has 1,100 robot tasks for coding agents, which are AI models that write and run programs
- The best agent gets **75%** when it can read the true state from the simulator, but **32%** with only the robot's cameras and sensors (Sim2Real)
- In real-time tests, where the world keeps moving while the agent thinks, Astra gets 34%, and the Claude and Gemini agents only **8.5%**
- So **seeing the true state** and **acting in time** are the hard parts



embodied-first-exam.ai leaderboard, batch run Oct 2–19, 2026. Success rate (%); each setting has its own test set (372 / 432 / 59 tests)

The Debate

- So what do all these results mean? Bigger models keep coming. Do we still need a harness?
- A CoRL 2026 workshop asks: is an AI agent on top of a trained robot model a **lasting design**? Or is it a **temporary scaffold** that bigger models will make unnecessary?

A lasting design or **A temporary scaffold**



Two Voices

- Here are two voices from this year

"Think of it as Claude acting as the puppeteer of a robot body"

Phillip Isola, MIT, in "Robot-Use Agents", September 2026

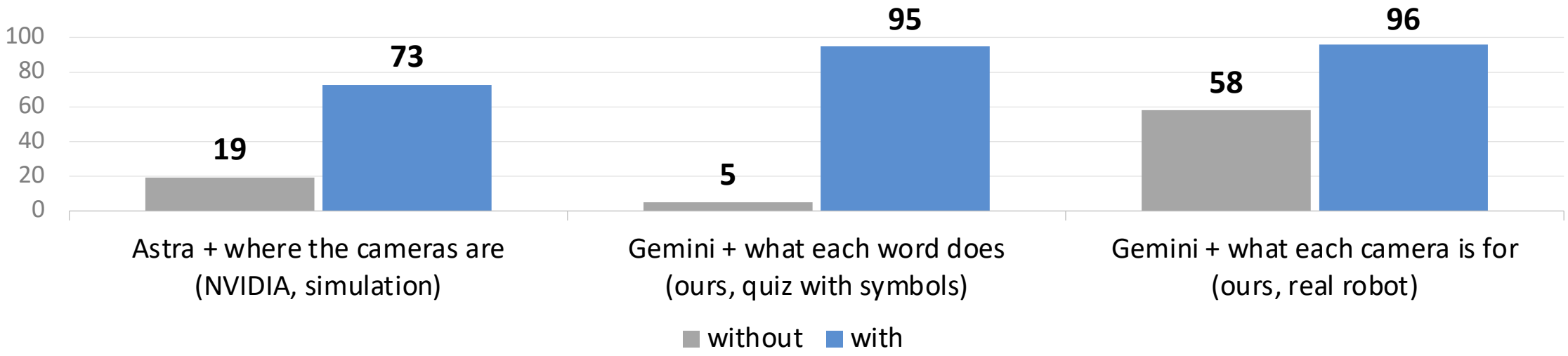
*"LLM robot control is progressing, but we're definitely not there yet. Low-level control for robotics still matters according to **Moravec's paradox**"*

Professor Jiafei Duan, NUS, on X, September 2026

- **Moravec's paradox**: what is easy for people, like grasping a cup, is hard for robots
- We agree with both: a VLM can steer a robot step by step, but **fine-grained tasks** like exact grasping and placing are still hard. Here is our view, in four insights

Insight 1: VLMs Know the World, Not Their Own Body

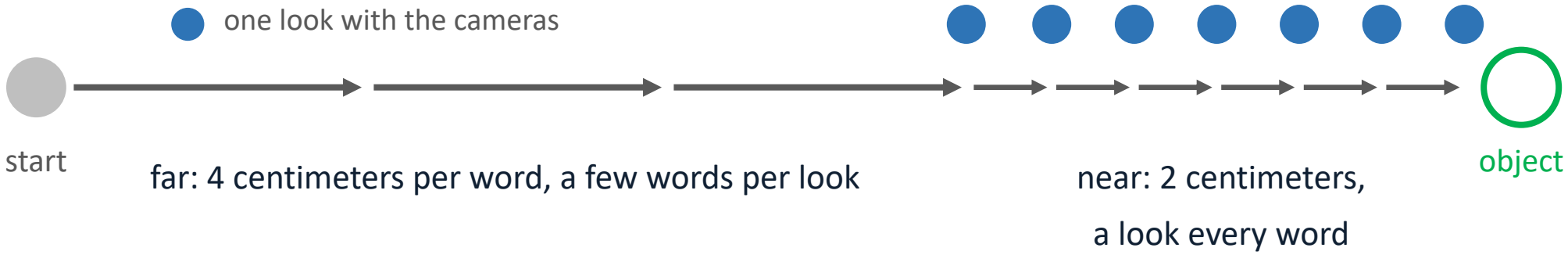
- VLMs learned the world from the internet. But they do not know the body they are using now: where its cameras are, what each action does, and how high the hand is
- When the harness tells them, success jumps. Told where the cameras are, GPT-6 Astra goes from 19% to **73%**. Given a rule sheet for our words, Gemini goes from 5% to **95%**
- So a large part of today's harness is a **description of the body**. Models trained with more robot data may learn some of it by themselves



Success rate (%); compare only within a pair. NVIDIA Astranomy (Oct 2026, camera and gripper geometry); our Fig. 10 and Fig. 9

Insight 2: Precision Comes from the Loop

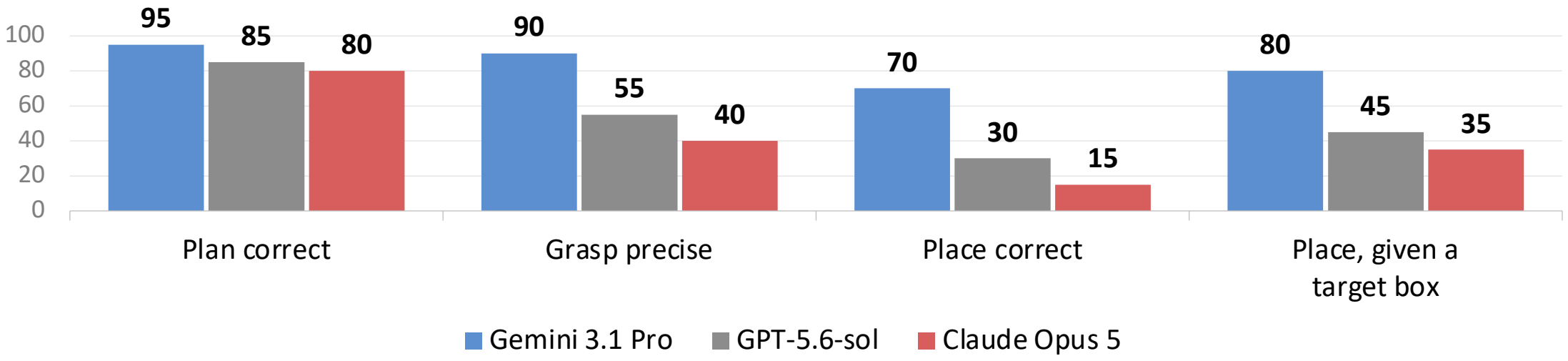
- In the game, the “robot” kept its eyes closed and never heard a number. Small steps, with a look after each one, were enough
- A robot is the same. Each step only needs the right direction, and the next look fixes the error
- But every look costs time. Our plugin tests show that looks matter most **near the object**. Far away, big steps and a few moves per look lose nothing
- Professor Duan wrote that thinking has to be adaptive. Our results add that **manipulation has to be adaptive** as well



Our plugin tests (Figure 9): adaptive steps reach 96% instead of 82% with 21% fewer steps, and a few moves per look keep 96% with 19% fewer model calls

Insight 3: Moravec's Paradox, in 2026

- Today's models plan well. The hard parts are **the last centimeter** and **acting in time**
- On our chess task, Gemini plans right 95% of the time, but places right only 70%. When the world keeps moving, the best agent in Embodied First Exam gets 34%
- With Gemini's plan, our small model goes from 10% to **70%**: **the big model thinks, and the small model moves**



Chess placement task, 20 trials (our Fig. 7b). Embodied First Exam, real-time setting: Codex + GPT-6 Astra 33.9%, Claude Code and Antigravity agents 8.5%. For the last centimeter, pointing models such as RoboPoint and MolmoAct are a promising direction

Insight 4: The Harness Gets Thinner, but Stays

- Back to the debate. Our answer: the harness will get thinner, but it will not disappear
- Some help is like a **driving teacher**: it explains the body to the model. Models trained with robot data may not need it
- Some help is like the **brakes and steering** of a car: safety limits and fast, precise motor control. Even the best driver still needs them

A temporary scaffold: will fade

where the cameras are

what each word does

what each camera is for

A lasting design: will stay

safety limits

fast, precise motor control

the same buttons for people and agents

Our Survey: From Computer-Use to Robot-Use

- Our survey, released last week, compares robots with computers
- **PAPAV** means **P**erceive, **A**nticipate, **P**lan, **A**ct and **V**erify: one loop for computers and robots
- Anticipate means guessing what an action will do. Verify means checking that it worked
- **Real World Robots are harder**: the camera cannot see everything, a broken cup has no undo, and the world keeps moving while the model thinks
- Of **64 benchmarks**, only **5** test anticipate, and only **3** test verify



Survey website
arXiv 2610.30108

Not Yet: Three Open Questions

- These results also show what does not work yet. Here are three open questions

Not yet	What we saw	Open question
Touch	Cameras cannot feel force. Astra inserted a plug 0 out of 11 times	Can touch become words too, like “the plug is stuck”?
Hands	Our action words are made for a gripper with two fingers	What are the action words for a hand with five fingers?
Long tasks	Our runs stop at 50 steps, and the model sees only its last 5 moves	How should a robot remember an hour of work? Text, frames, video?

Ongoing: RUIBench, a Robot-Use Interface Benchmark

- One problem came up many times today: each work tests its own interface, on its own tasks
- So the results are hard to compare. For example, Embodied First Exam uses only code
- Our ongoing work is **RUIBench**. It keeps the model, the robot and the tasks the same
- **Only the interface changes.** It is in progress, and we hope to share it soon ☺

Interfaces people use today	The model says	Example
Action words	MV_LEFT, GRASP	Show-Harness
Positions	move_to(x, y, z)	RoboCurve
Code	pick(cup); place(cup, plate)	CaP-X
Call a VLA	“put the cup on the plate”	Gemini Robotics 2

Takeaways

- A **VLM can play a real robot** without being trained on it: it says one small action word at a time, and looks again after every step
- The same words also train **a small open model** in under 2 hours, and it runs much faster
- VLMs know the world but not their own body, and the **last centimeter is still hard**
- The harness will get thinner, but **safety limits and fast motor control** will stay
- Everything is **open**: the code, models, data and GUMI are on our website

We hope Show-Harness is a step toward robots that **any VLM agent** can use, **easily** and **openly**

Website

Thank you!

I would be happy to take your questions

Code · Models

Data · GUMI



Joint work with Zechen Bai, Zhijun Cao, Wenzheng Zeng, Kevin Qinghong Lin, Yiqi Lin, Guoqiang Liang, Kevin Yuchen Ma, Qiming Huang and Mike Zheng Shou

Copyright©Mike Shou, NUS

Backup: Questions

B1. Why is GPT-6 Astra not in the paper?

B2. Is the comparison with π0.5 and GR00T fair?

B3. Why not MolmoAct2 as a baseline?

B4. How is this different from Guava?

B5. Isn't step-by-step control slow?

B6. What about contact-rich or dexterous tasks?

B7. How is this different from RT-H language motions?

B8. How is safety handled?

B9. Could the frontier model have seen these tasks?

B10. What data and training do you use?

B11. What do π0.5's failures look like?

B12. Can it rotate to unseen angles?

B13. Can it learn a task from one video?

B14. How small can the model be?

B15. Does it work outside the lab?

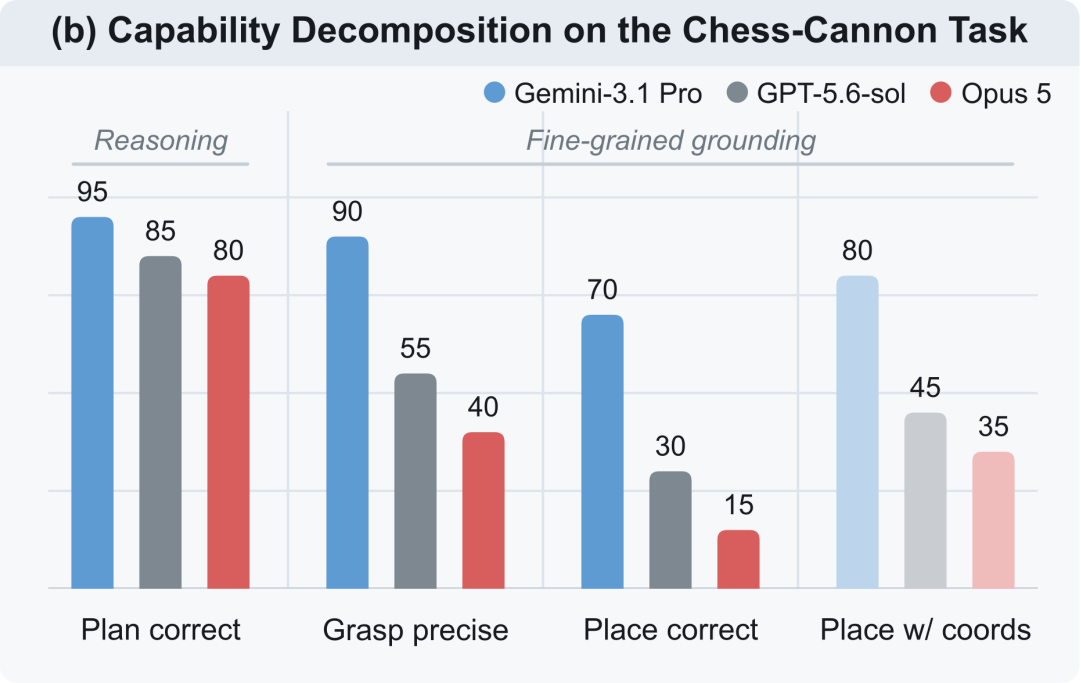
B16. Where are the full results? (Table 2)

B17. What does each plugin add? (Fig. 9)

B18. What else can it do? (Fig. 6)

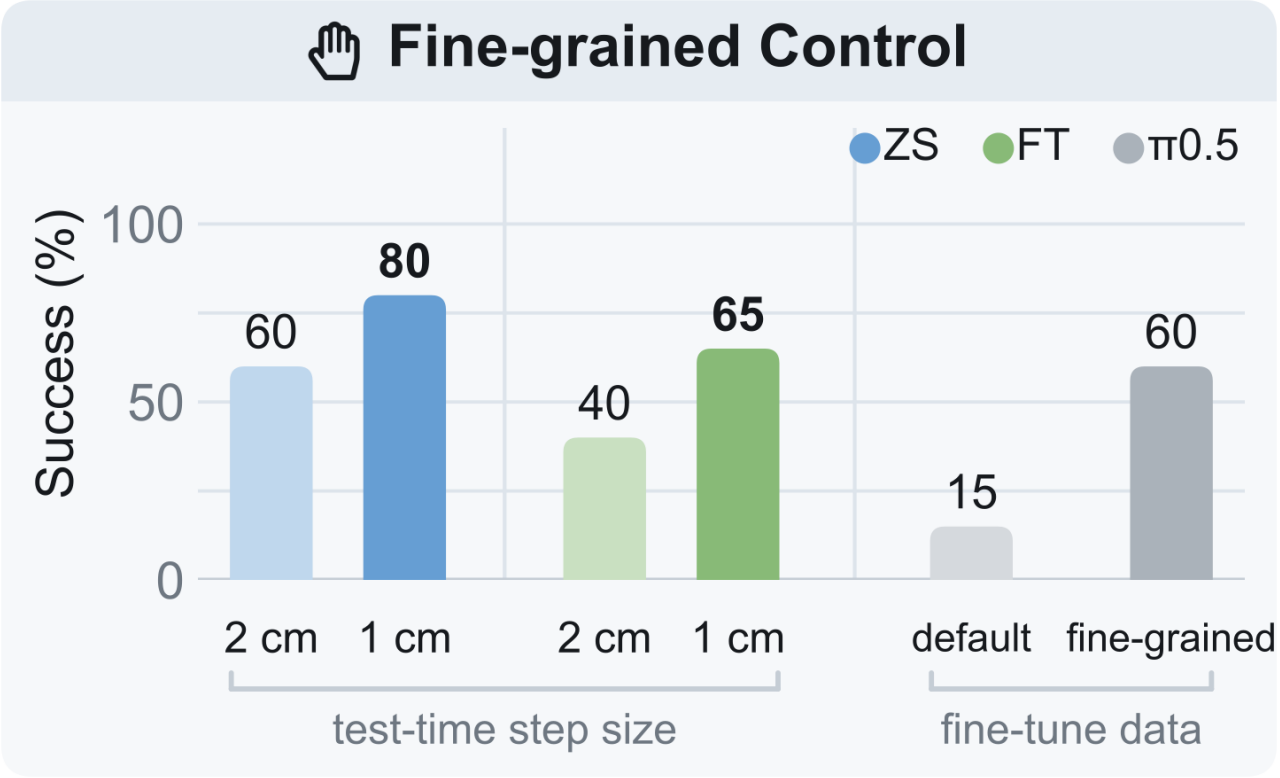
Q: Why is GPT-6 Astra not in the paper?

- Our experiments ended before Astra came out. Our paper came out 6 days after it
- The harness does not depend on the model. Using Astra is one more setting
- Others show that Astra gains a lot from a harness: 19% to 73% with geometry in the prompt
- We expect Astra to help most where our models fail: **exact grasping and placing**



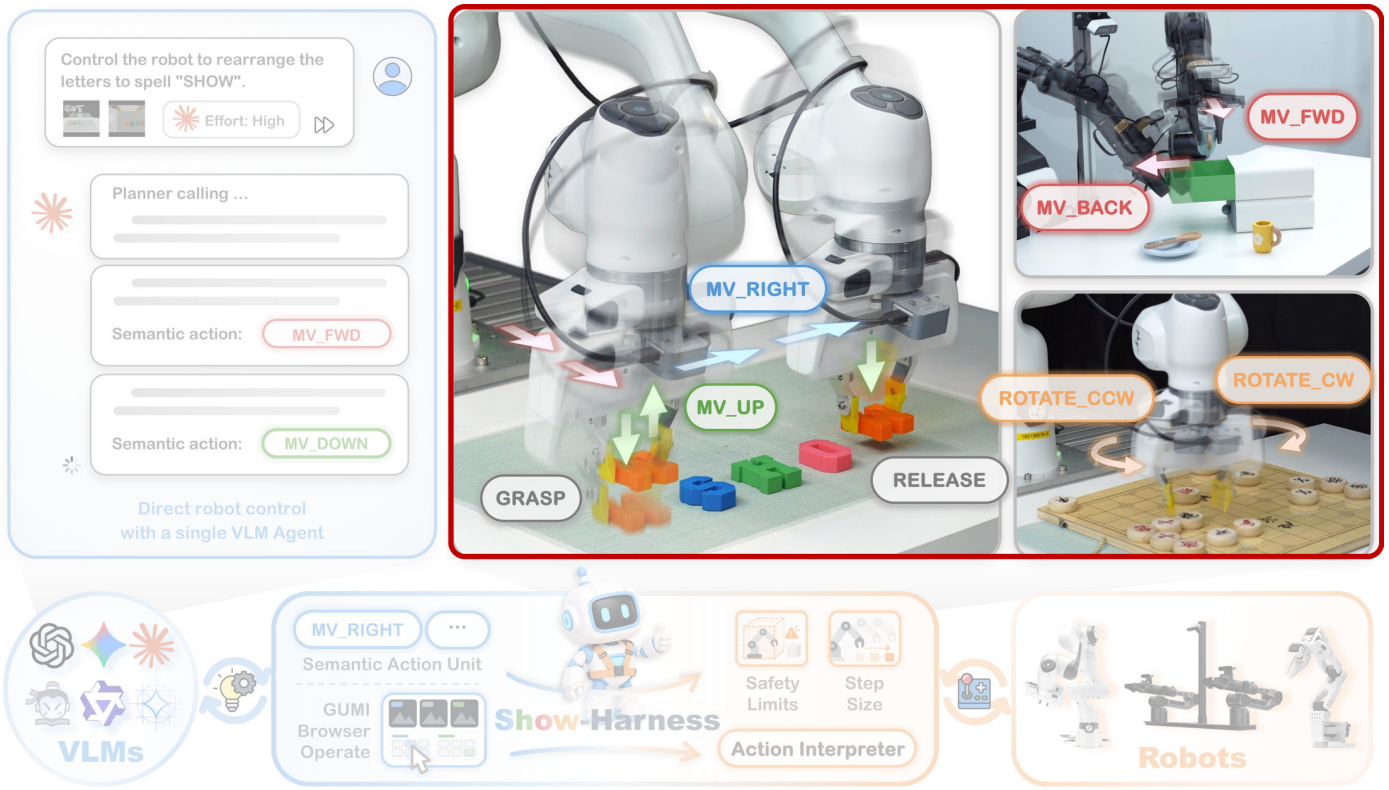
Q: Is the comparison with π0.5 and GR00T fair?

- Both were fine-tuned on **the same 164 real demos**, as continuous gripper moves
- Our demos are stop-and-go steps, which may not suit VLAs. Smooth demos are future work
- With more data, VLAs get much stronger. Our claim is only about **the same small data**



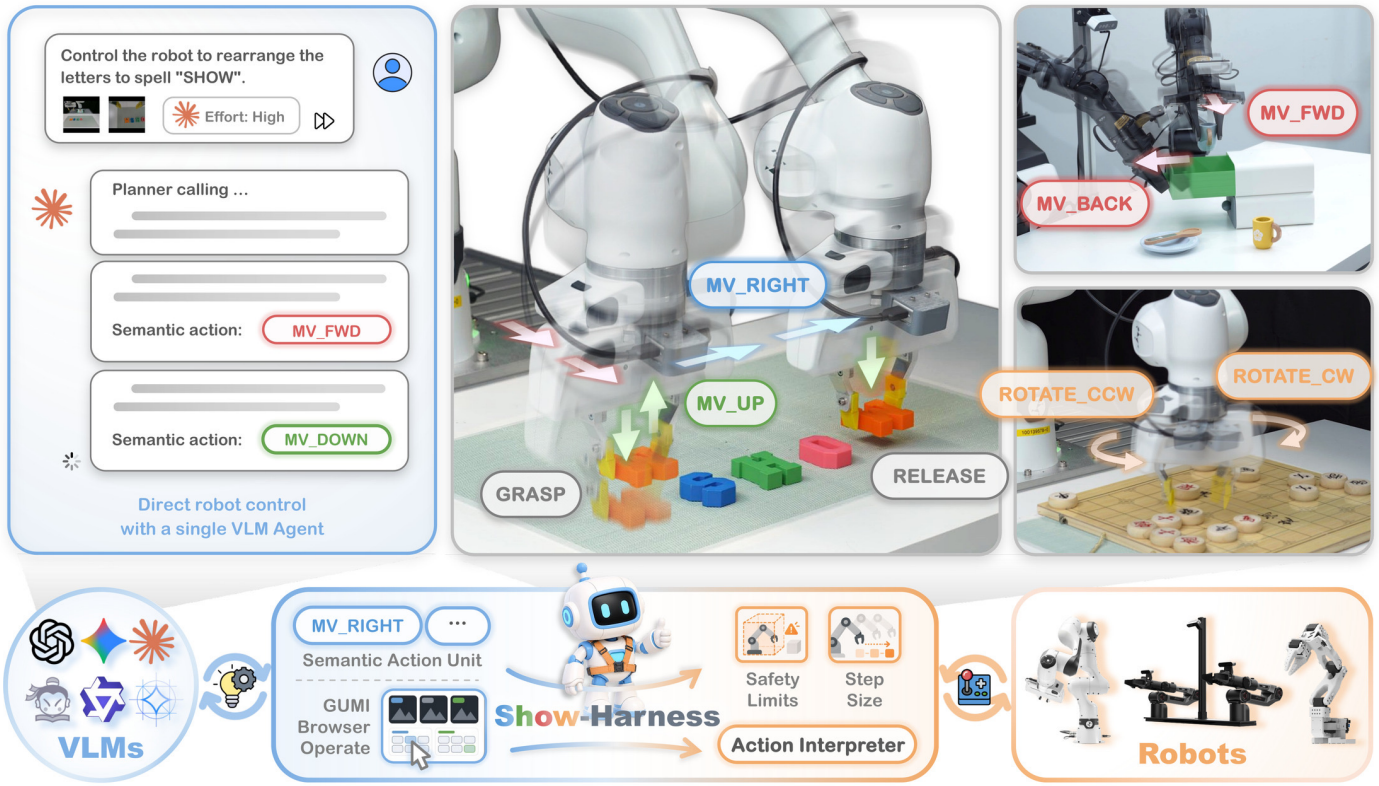
Q: Why not MolmoAct2 as a baseline?

- MolmoAct2, from May 2026, is a fully open **action reasoning model**, built for real-world use
- We compared π0.5 and GR00T, trained on our data. MolmoAct2 is a strong next baseline
- Its visual traces and our semantic actions share one goal: **actions people can read**



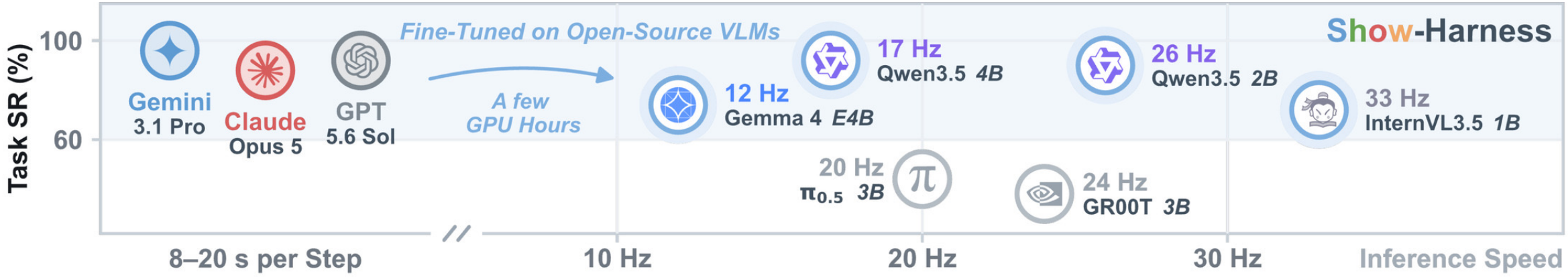
Q: How is this different from Guava?

- Guava, from June, distills perception, planning and control modules into a small open model
- In Show-Harness, the VLM itself makes **every small move**, with a small set of action words
- The same words serve people in GUMI, frontier VLMs and our small model, on two robots



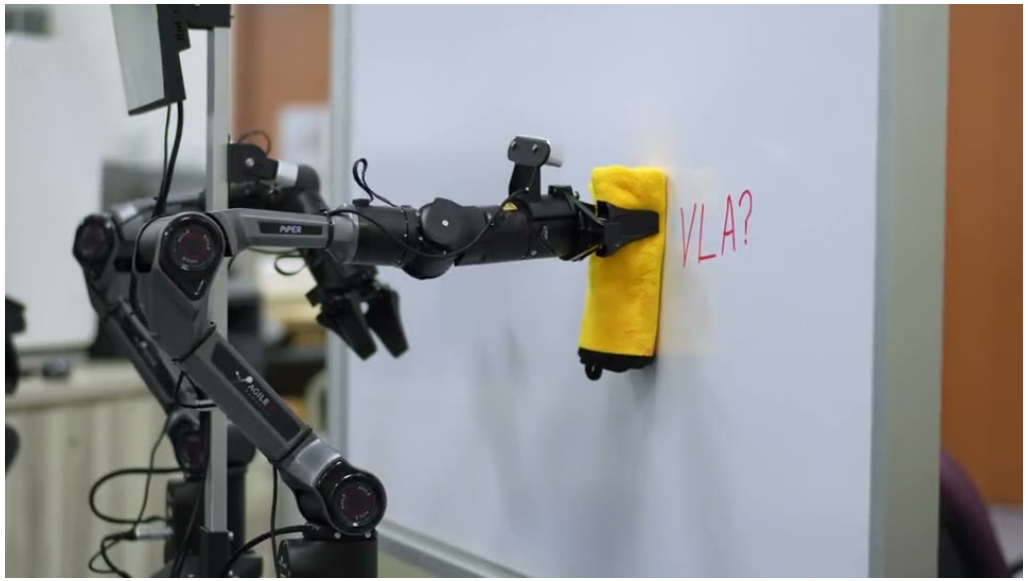
Q: Isn’t step-by-step control slow?

- Frontier VLMs take **8 to 20 seconds per step**, and an episode takes about 30 steps
- Action chunking makes 19% fewer model calls than no chunking, with the same 96% success
- The adaptive step takes 21% fewer steps than 2-centimeter steps alone, and success goes from 82% to 96%
- Our small fine-tuned model makes about **26 steps per second** on one RTX 5090
- The robot videos in this talk play at 2 times speed



Q: What about contact-rich or dexterous tasks?

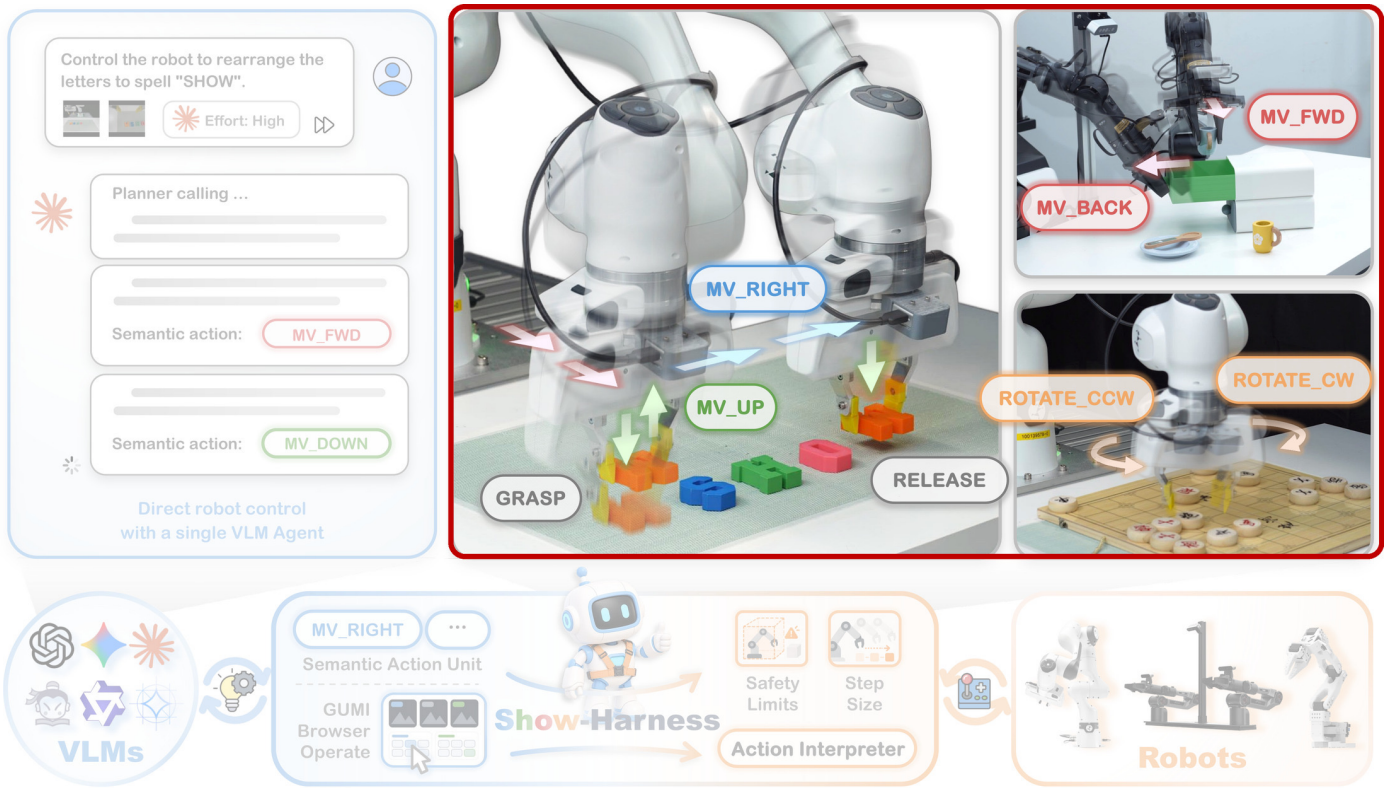
- Not yet. We use parallel grippers, cameras and simple body state
- Folding and wiping work in demos, but tight insertion needs **touch and force**
- Next, we want dexterous hands, humanoids and touch sensing





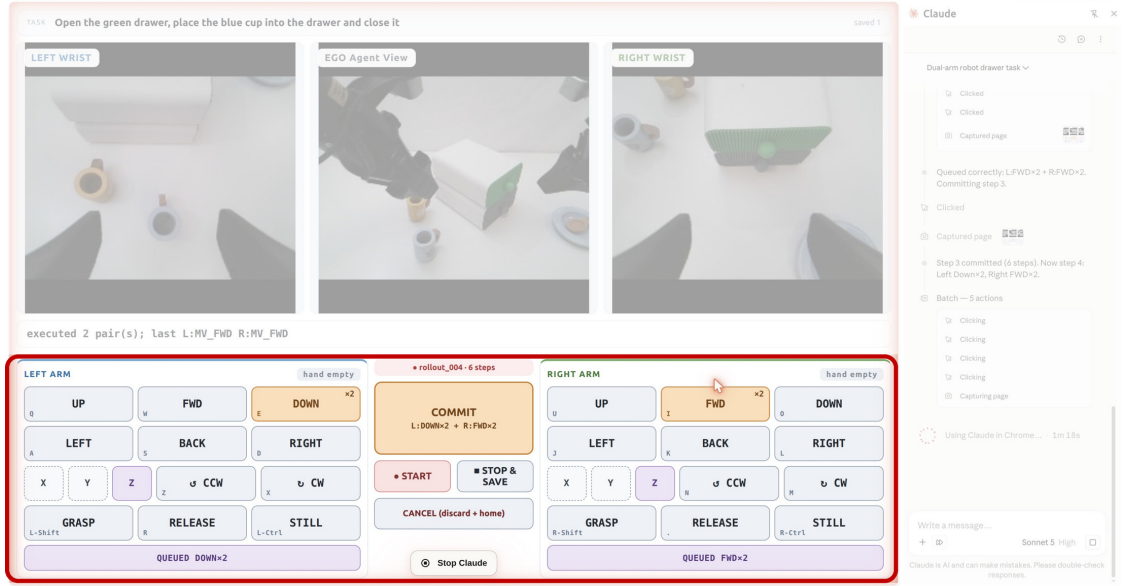
Q: How is this different from RT-H language motions?

- RT-H, from 2024, predicts language motions like “move arm forward” inside a trained VLA
- In Show-Harness, a frontier VLM uses the words **zero-shot**, and a fixed interpreter runs them
- The same words also serve people and agents in GUMI, and small models after fine-tuning



Q: How is safety handled?

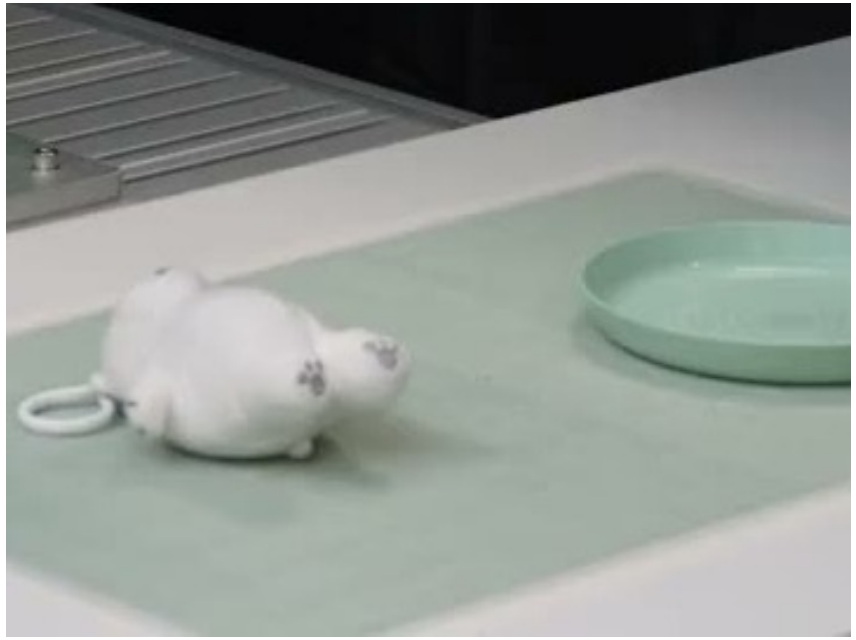
- Every action is a **readable word**, and we log it at each step
- The interpreter blocks moves outside the workspace or below the table height
- Each step moves only 2 to 4 centimeters, so one wrong decision is small and can be fixed
- Action chunking runs a few steps without a new look, but only while the object is still far
- In GUMI, a person can take over at any time

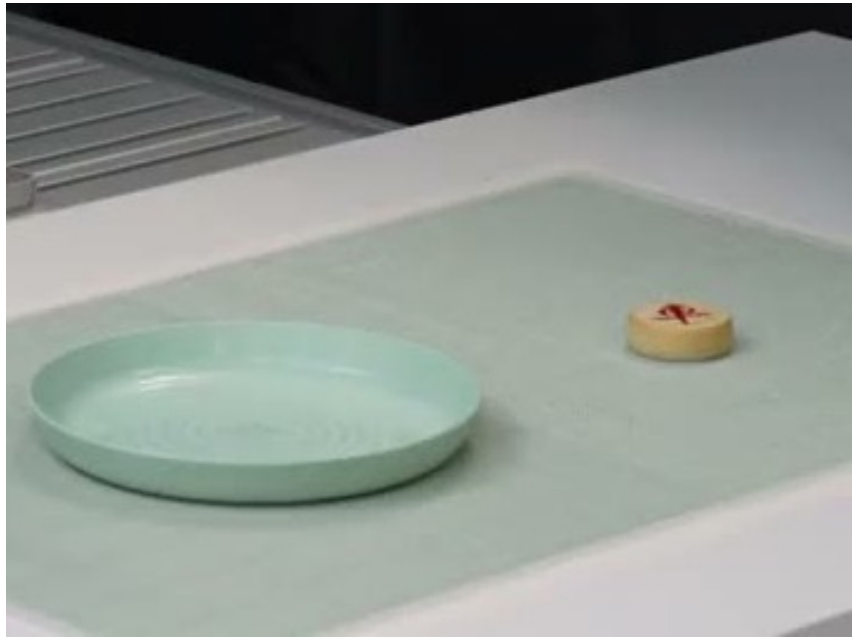


← Back
Index

Q: Could the frontier model have seen these tasks?

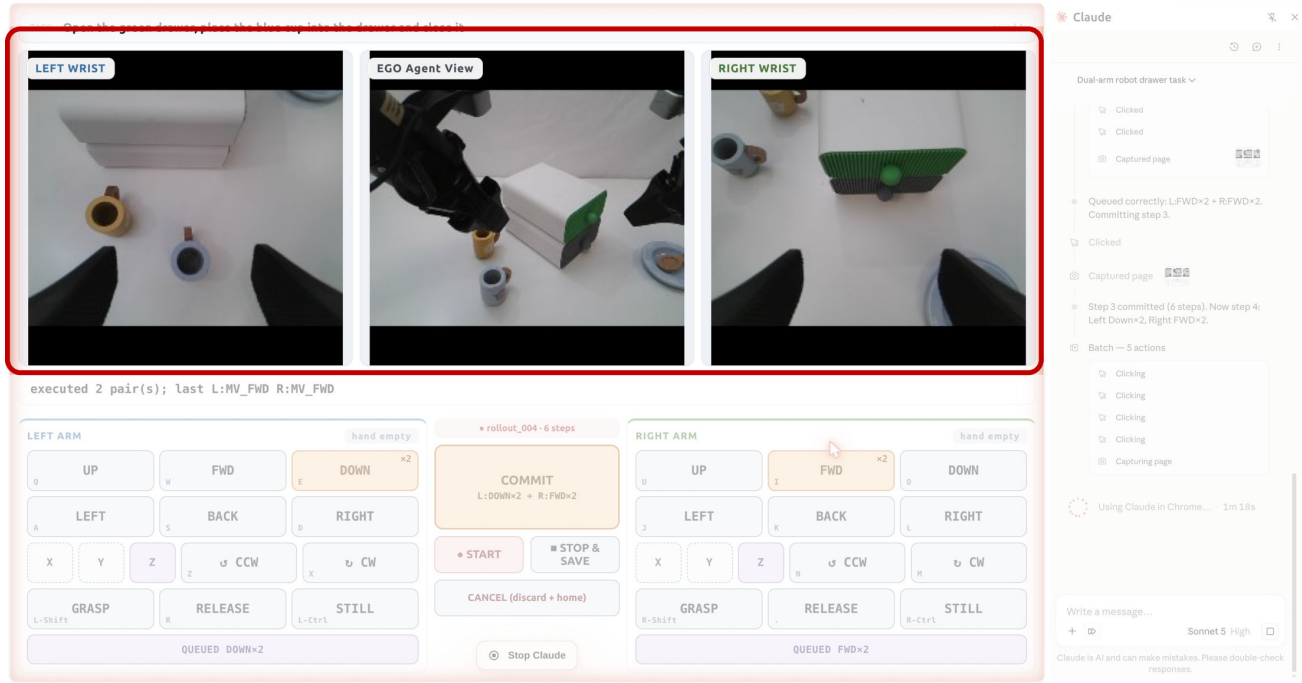
- The tasks, objects and scenes are our own. Two objects are held out from fine-tuning
- Closed models may have seen robot data, as Professor Duan asked. We cannot rule this out
- So we also report the **open fine-tuned model**, whose fine-tuning data is fully known





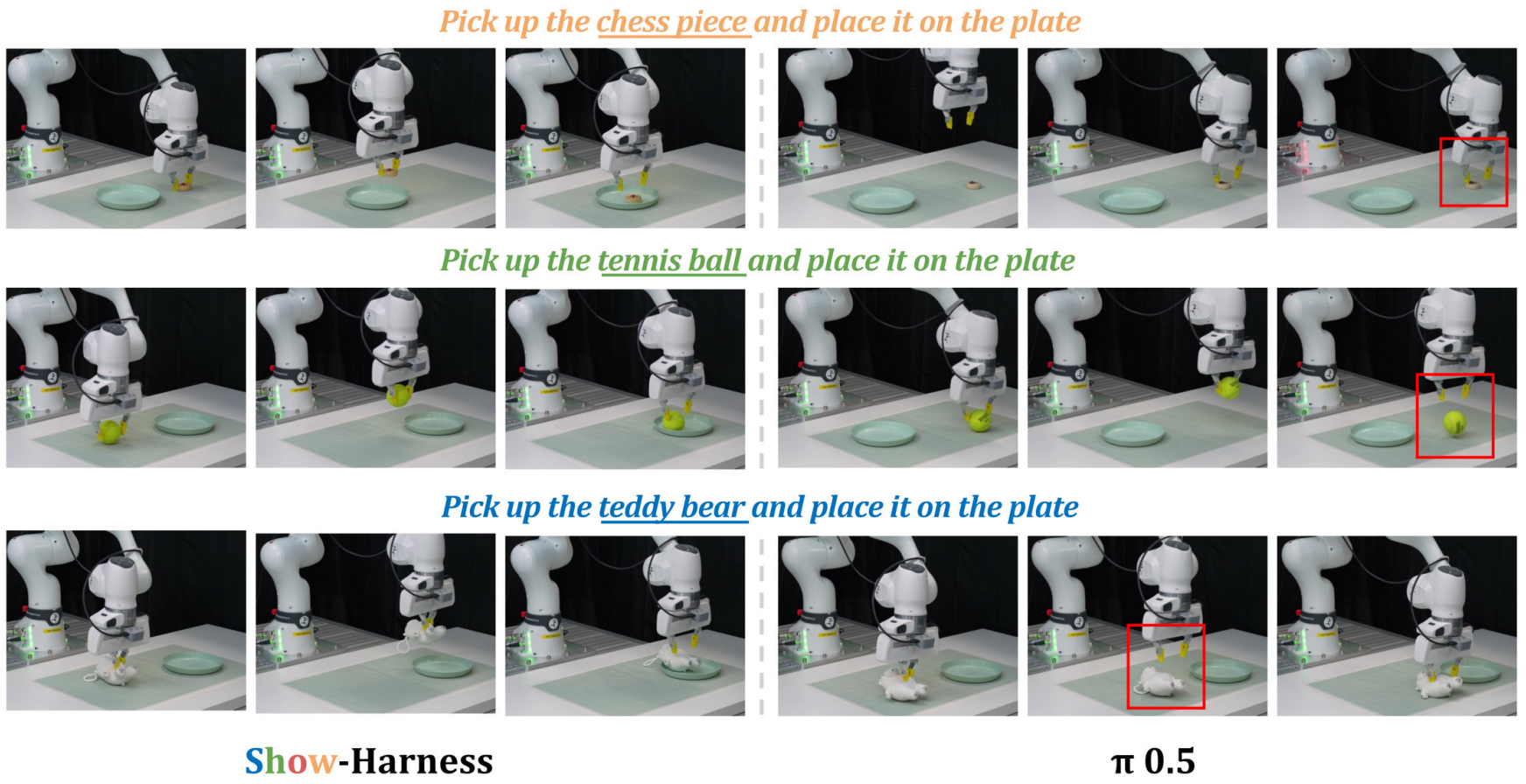
Q: What data and training do you use?

- Real demos from GUMI: 101 episodes on Franka and 63 on AgileX, with 2-centimeter steps
- Simulation: 230 episodes, about 13,500 steps, in ManiSkill and RoboLab
- We fine-tune Qwen 3.5 with 2 billion parameters, using LoRA rank 64, for 40 epochs
- It takes under 2 hours on one H200 GPU. A 24-gigabyte GPU is enough



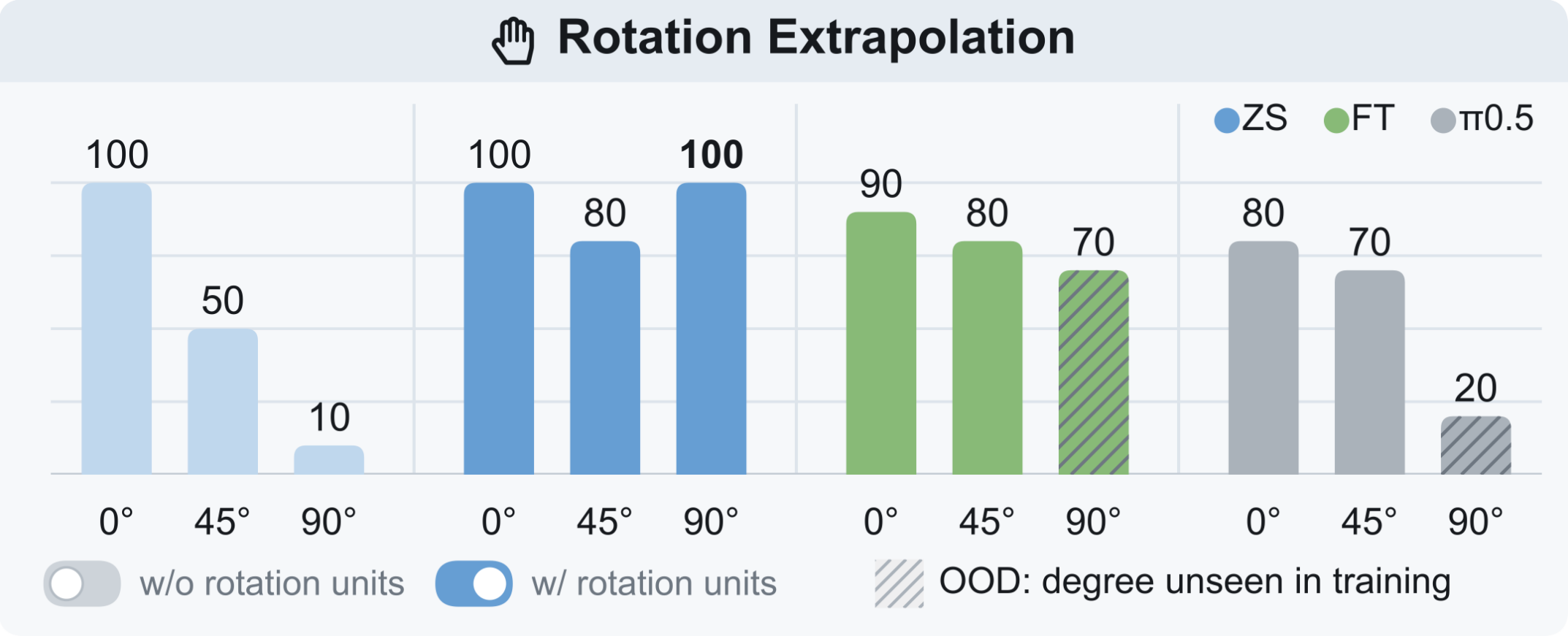
Q: What do π0.5's failures look like?

- Both use the same demos: π0.5 learns continuous motions, and we learn the words
- π0.5 often **hits or drops** the object, as the red boxes show



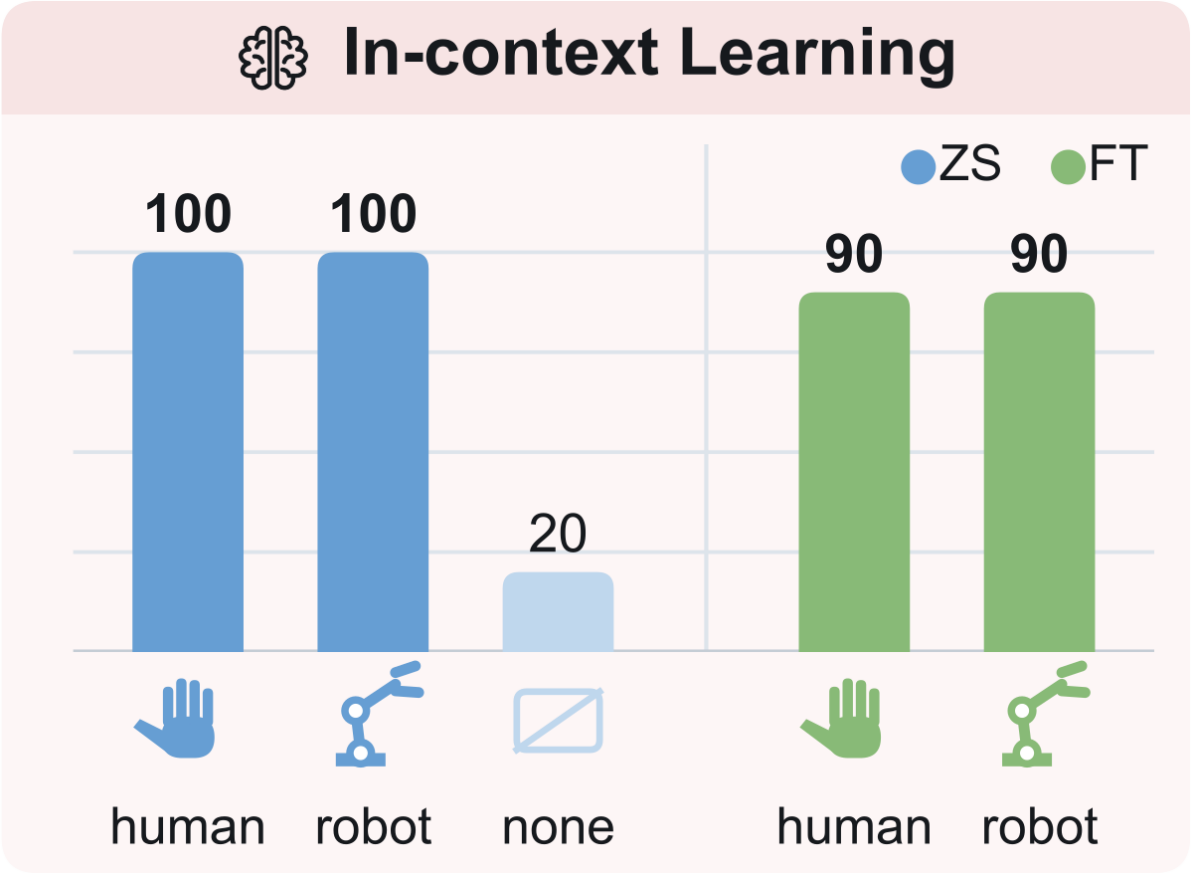
Q: Can it rotate to unseen angles?

- Each ROTATE step turns **15 degrees**. In training, the carrots were only at 0 and 45 degrees
- At an unseen **90 degrees**, fine-tuned gets **70%**, and π0.5 only **20%**



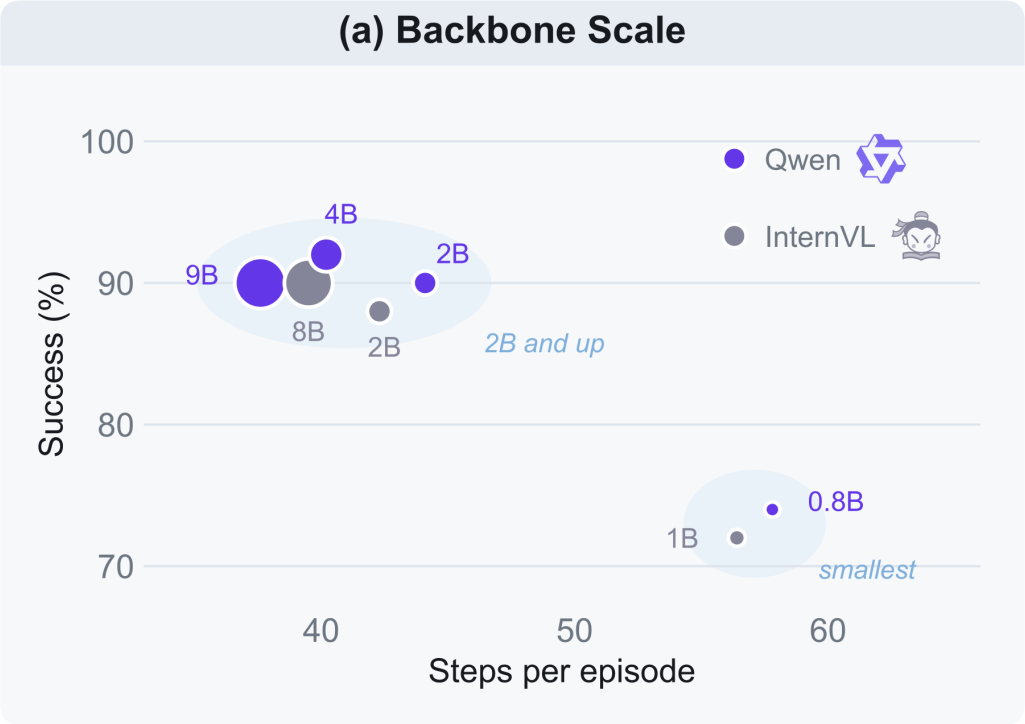
Q: Can it learn a task from one video?

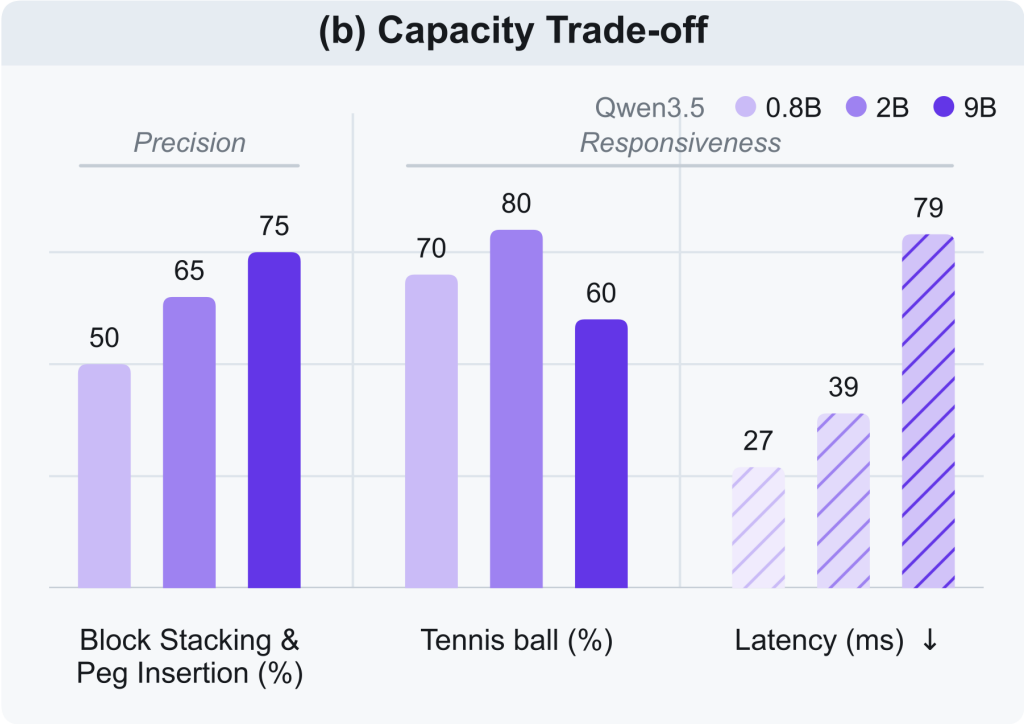
- The task is “tidy up”, in the order shown in **one video** of a human or a robot
- With zero-shot and no video, it gets 20%. With a video, it gets **100%**, 20 out of 20



Q: How small can the model be?

- A **2-billion** model is enough for pick-and-place. Smaller ones shake near the target
- Bigger helps precision: on stacking and peg insertion, 0.8 billion gets **50%** and 9 billion **75%**
- Our 2-billion model is twice as fast as the 9-billion one, and better on a rolling ball





Q: Does it work outside the lab?

- It can erase a whiteboard, put pens into a holder, and cut a cake
- These are demo videos: the paper has no success rates for these tasks



erase the whiteboard



pens into a holder



cut a cake

Q: Where are the full results? (Table 2)

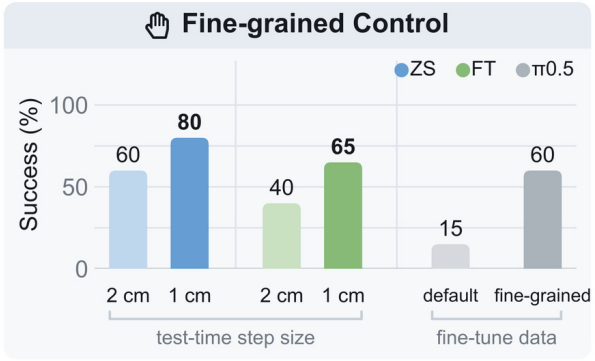
Setting	VLA		VLA-centric agent		Code-as-policy agent		Show-Harness	
	$\pi_{0.5}$	GR00T	H-VLA	G-VLA	CaP-X	RATS	ZS	FT
Cross-Task *10 object–receptacle tasks; †: unseen in fine-tuning; 10 trials / task*								
Block → Plate	6 / 10	5 / 10	7 / 10	2 / 10	5 / 10	6 / 10	10 / 10	10 / 10
Banana → Plate	7 / 10	7 / 10	8 / 10	3 / 10	8 / 10	8 / 10	10 / 10	10 / 10
Tennis → Plate	3 / 10	3 / 10	3 / 10	1 / 10	4 / 10	6 / 10	8 / 10	8 / 10
Teddy† → Plate	5 / 10	4 / 10	5 / 10	1 / 10	4 / 10	6 / 10	10 / 10	8 / 10
Chess† → Plate	1 / 10	0 / 10	3 / 10	0 / 10	2 / 10	4 / 10	10 / 10	9 / 10
Block → Bowl	5 / 10	4 / 10	6 / 10	2 / 10	5 / 10	5 / 10	10 / 10	10 / 10
Banana → Bowl	6 / 10	6 / 10	7 / 10	2 / 10	7 / 10	8 / 10	6 / 10	7 / 10
Tennis → Bowl	2 / 10	2 / 10	3 / 10	1 / 10	3 / 10	5 / 10	7 / 10	8 / 10
Teddy† → Bowl	3 / 10	4 / 10	5 / 10	1 / 10	4 / 10	5 / 10	8 / 10	7 / 10
Chess† → Bowl	1 / 10	0 / 10	3 / 10	0 / 10	2 / 10	4 / 10	10 / 10	9 / 10
Average (%)	39.0	35.0	50.0	13.0	44.0	57.0	89.0	86.0
Cross-Environment *2 tasks (Block or Banana → Plate); 10 trials / task*								
Background	11 / 20	10 / 20	15 / 20	4 / 20	12 / 20	14 / 20	20 / 20	18 / 20
Lighting	9 / 20	9 / 20	14 / 20	5 / 20	11 / 20	13 / 20	20 / 20	19 / 20
Viewpoint	10 / 20	7 / 20	10 / 20	2 / 20	10 / 20	12 / 20	20 / 20	19 / 20
Distractors	10 / 20	8 / 20	12 / 20	1 / 20	9 / 20	13 / 20	20 / 20	19 / 20
Sim-to-real	0 / 20	0 / 20	–	–	–	–	–	13 / 20
Average (%)	40.0	34.0	63.8	15.0	52.5	65.0	100.0	88.0
Cross-Embodiment *5 Plate tasks; 10 trials / task on each arm*								
Franka (7-DoF)	22 / 50	19 / 50	26 / 50	7 / 50	23 / 50	30 / 50	48 / 50	45 / 50
AgileX (6-DoF)	19 / 50	17 / 50	23 / 50	4 / 50	20 / 50	22 / 50	45 / 50	42 / 50
Average (%)	41.0	36.0	49.0	11.0	43.0	52.0	93.0	87.0

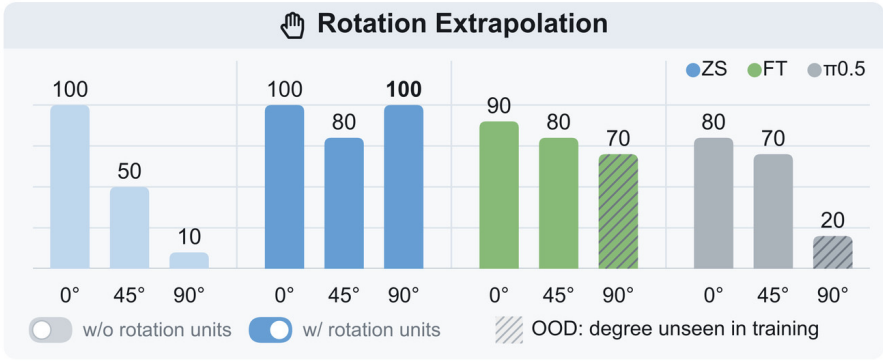
Q: What does each plugin add? (Fig. 9)

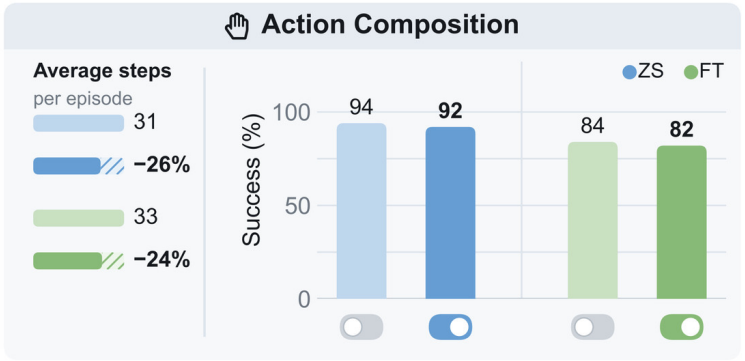


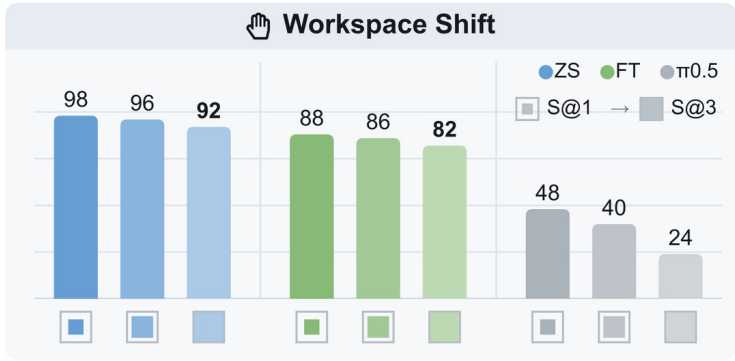
← Back
Index

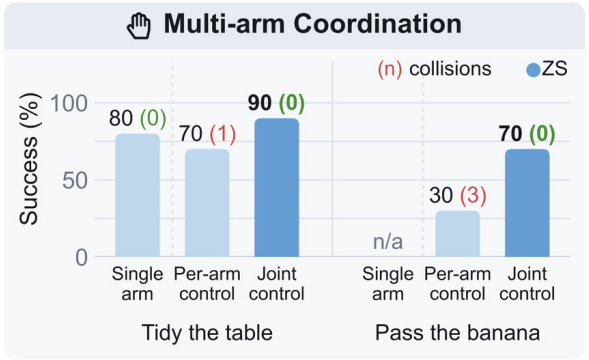
Q: What else can it do? (Fig. 6)

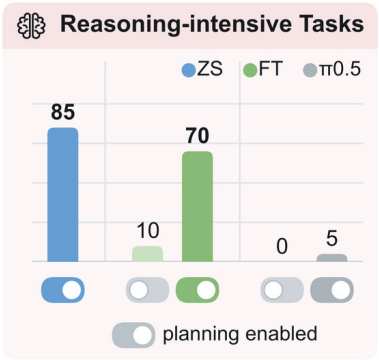


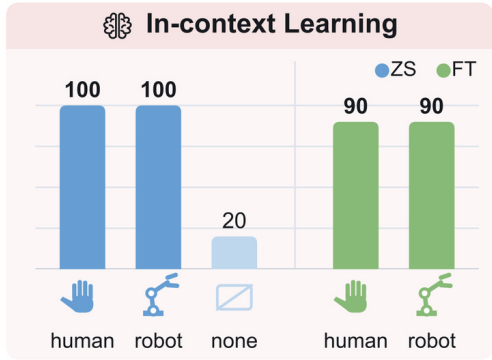












← Back

Index